



TRAKYA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

YAPAY SİNİR AĞLARINA GİRİŞ
PROJE RAPORU

PROJE ADI:

VocaRig

PROJE EKİBİ:

1221602076 – Eren KAÇAR

1221602609 – Yakup Can KILIÇ

1221602090 – Hasan Furkan DUZMAN

1221602089 – Kaan YILDIZ

1221602063 – Kerem Berke ŞEN

PROJE DANIŞMANI:

Dr. Öğr. Üyesi Turgut DOĞAN

Edirne - 2026

İÇİNDEKİLER

1. GİRİŞ & PROJENİN TANITIMI	1
1.1. Projenin Amacı.....	1
1.2. Projenin Kapsamı	2
1.3. Benzer Çalışmalar ve Yenilikçi Yön.....	2
1.4. Raporun Düzeni.....	3
2. GEREKSİNİM ANALİZİ	4
2.1. Problem Tanımı.....	4
2.2. Hedef Kitle ve Kullanıcı Rollerı	4
2.3. Platform ve İşletim Sistemi Gereksinimleri	5
2.4. Teknoloji Gereksinimleri	6
2.5. Kaynak Kısıtları ve Çözüm Yaklaşımları	7
2.6. İşlevsel ve İşlevsel Olmayan Gereksinimler	8
2.7. Sistem Bağlamı ve Dış Bağımlılıklar	9
3. TASARIM.....	11
3.1. Genel Sistem Mimarısı.....	11
3.2. Veri Tasarımı.....	13
3.3. Model Tasarımı	14
3.4. Çalışma Zamanı ve Arayüz Tasarımı.....	17
3.5. Eğitim ve Dışa Aktarım Tasarımı	19
3.6. API ve Veri Arayüzü Tasarımı.....	19
3.7. ARKit ve FaceRig Birleşim Tasarımı	21
4. GELİŞTİRME	22
4.1. Proje Dizin Yapısı	22
4.2. Ses Özellik Çıkarımı	23

4.3. Sentetik Veri Üretimi	23
4.4. Model Eğitimi	25
4.5. Gerçek Zamanlı Çıkarım	26
4.6. Web Arayüzü.....	28
4.7. Kurulum ve Çalıştırma Yönergeleri.....	30
4.8. Eğitim Artefaktları ve Dosya Yönetimi	30
4.9. Geliştirme Süreci ve Git Geçmişi.....	31
4.10. Önemli Kod Davranışları	32
5. TEST ve DOĞRULAMA.....	34
5.1. Test Stratejisi.....	34
5.2. Birim ve Sözleşme Testleri	34
5.3. UI/API Doğrulaması	35
5.4. Eğitim Sonuçları.....	36
5.5. Bulguların Değerlendirilmesi	38
5.6. Kabul Senaryoları.....	39
5.7. Değerlendirme Ölçütleri.....	39
5.8. Testlerin Genişletilmesi İçin Öneriler	40
6. SONUÇ	42
6.1. Genel Sonuç	42
6.2. Sınırlılıklar	42
6.3. Gelecek Çalışmalar	43
KAYNAKLAR.....	44

1. GİRİŞ & PROJENİN TANITIMI

VocaRig, konuşma sesinden dudak, çene ve ağız bölgesine ait ARKit uyumlu blendshape değerleri üretmek amacıyla geliştirilen bir yapay sinir ağı projesidir. Proje, ses sinyali zaman içinde işleyerek konuşmanın artikülasyon hareketlerini tahmin eder ve bu tahminleri dijital karakterlerin yüz animasyonlarında kullanılabilecek sayısal kontrol değerlerine dönüştürür. Bu yönüyle VocaRig, klasik yüz ifadesi yakalama sistemlerinden farklı olarak konuşma kaynaklı ağız hareketlerini bağımsız bir üretim hattı olarak ele alır.

Projenin temel fikri, yüz ifadesi ve konuşma artikülasyonunun her zaman aynı problem olmamasıdır. Kaş, göz, yanak ve genel mimik hareketleri bir karakterin duygusal durumunu ifade ederken; çene açılması, dudak büzülmesi, dudak kapanması ve ağız kenarı hareketleri doğrudan konuşmanın fonetik yapısıyla ilişkilidir. VocaRig bu ayrımı temel alır. FaceRig benzeri bir sistemin genel ifade kanallarını yönetebileceği, VocaRig'in ise konuşma kaynaklı dudak ve çene kanallarını üretebileceği bir yapı hedeflenmiştir.

Proje dizinindeki uygulama incelendiğinde sistemin yalnızca bir model dosyasından oluşmadığı görülmektedir. VocaRig; sentetik veri üretimi, ses özellik çıkarımı, GRU tabanlı model eğitimi, ONNX dışa aktarımı, FastAPI tabanlı servis katmanı, WebSocket destekli gerçek zamanlı çıkarım, Three.js tabanlı 3B ön izleme ve test altyapısını birlikte içeren bütünlüklü bir yazılım projesidir. Bu nedenle rapor, yalnızca model mimarisini değil, bu mimarinin uçtan uca yazılım sistemi içinde nasıl kullanıldığını da açıklamaktadır.

Projenin kaynak kod deposu GitHub üzerinde “<https://github.com/MC-Oruc/VocaRig>” adresinde yer almaktadır. Bu depo; kaynak kodları, veri setini, eğitim betiklerini, arayüz dosyalarını, testleri, konfigürasyonları ve model artefaktlarını içermektedir.

1.1. Projenin Amacı

Projenin amacı, ham ses verisini kullanarak dijital bir karakterin konuşmaya uygun dudak hareketlerini otomatik üretmektir. Bu amaç doğrultusunda sistem, ses sinyalinden log-mel tabanlı özellikler çıkarır, bu özellikleri geçmiş dudak durumu ve zaman/stil bilgisiyle birlikte sinir ağına verir, ağın ürettiği 21 boyutlu dudak kontrol vektörünü tam ARKit blendshape sözlüğüne yerleştirir ve sonucu arayüzde görselleştirir.

Bu hedef, özellikle oyun geliştirme, sanal avatar, yayıncılık, animasyon ön üretimi, gerçek zamanlı karakter kontrolü ve konuşma tabanlı yüz animasyonu gibi alanlarda anlamlıdır. Manuel dudak senkronizasyonu zaman alıcıdır. Tam yüz yakalama sistemleri ise her zaman

uygun donanım gerektirir. VocaRig, doğrudan ses dosyası veya mikrofon akışı üzerinden çalışabildiği için daha erişilebilir bir animasyon üretim yaklaşımı sunar.

1.2.Projenin Kapsamı

VocaRig projesi aşağıdaki ana bileşenleri kapsar:

- Sentetik konuşma-artikülasyon veri kümesi üretimi.
- WAV ses verisinin mono forma getirilmesi ve örnekleme hızının dönüştürülmesi.
- Log-mel spektrogram tabanlı özellik çıkarımı.
- 11 bağlam karesi ve 80 mel bandı ile çalışan nedensel ses penceresi tasarımı.
- 21 adet dudak/çene kontrolünü tahmin eden GRU tabanlı model.
- Model çıktısının 52 elemanlı ARKit blendshape sözlüğüne dönüştürülmesi.
- Baked, no-baked ve canlı mikrofon çıkarım modları.
- Eğitim, metrik izleme, model seçimi, benchmark ve ONNX dışa aktarım arayüzleri.
- Testler ile sözleşme, eğitim, veri ve UI/API davranışlarının doğrulanması.

Proje kapsamı dışında kalan konu ise tam yüz ifadesi üretimidir. Sistem kaş, göz ve genel mimik kanallarını doğrudan üretmek için tasarlanmamıştır. Bu kanalların FaceRig benzeri başka bir sistem tarafından sağlanması ve VocaRig çıktısıyla birleştirilmesi hedeflenmiştir.

1.3. Benzer Çalışmalar ve Yenilikçi Yön

Ses tabanlı dudak senkronizasyonu uzun süredir animasyon ve oyun alanlarında kullanılan bir problemdir. Geleneksel yaklaşımlarda konuşma fonemlere ayrılır, her fonem için bir viseme karşılığı seçilir ve zaman çizelgesine göre ağız şekilleri canlandırılır. Bu yaklaşım anlaşılır ve kontrol edilebilir olmakla birlikte, konuşmanın akıcılığı ve kişisel ağız hareketi farklılıklarını yakalamakta sınırlı kalabilir.

VocaRig'in yenilikçi yönü, ses özellikleri ile önceki dudak durumunu birlikte kullanan nedensel bir GRU modeliyle sürekli değer üretmesidir. Model her karede yalnızca hedef ağız pozunu değil, önceki kareden yeni kareye geçişin büyüklüğünü de sınırlayan bir çıktı yapısı kullanır. Bu sayede ağız hareketlerinin ani sıçramalar yerine daha kontrollü ilerlemesi hedeflenmiştir.

Projenin bir diğer önemli yönü, eğitim ve çıkarım süreçlerinin aynı çalışma alanında birleştirilmesidir. Kullanıcı yalnızca komut satırından model eğitmek zorunda değildir; arayüz

üzerinden veri seti seçebilir, eğitim parametrelerini düzenleyebilir, eğitim durumunu izleyebilir, model seçebilir, çıkarım yapabilir ve performans ölçebilir. Bu yapı, projeyi yalnızca bir araştırma kodu olmaktan çıkarıp kullanılabilir bir laboratuvar uygulamasına dönüştürmektedir.

1.4. Raporun Düzeni

Bu raporda önce problem ve gereksinimler açıklanmış, ardından sistemin mimarisi, veri hattı, model tasarımı ve arayüz yapısı anlatılmıştır. Geliştirme bölümünde proje dizinindeki gerçek modüller temel alınarak uygulama ayrıntıları verilmiştir. Test ve doğrulama bölümünde mevcut testler, eğitim çıktıları ve doğrulama bulguları sunulmuştur. Sonuç bölümünde ise projenin kazanımları, sınırlılıkları ve gelecek geliştirme önerileri değerlendirilmiştir.

Şekil ve çizelgeler kılavuza uygun olarak bölüm numarasıyla verilmiştir. Şekil açıklamaları şekillerin altında, çizelge açıklamaları çizelgelerin üstünde yer almaktadır.

2. GEREKSİNİM ANALİZİ

Bu bölümde VocaRig'in çözmeyi amaçladığı problem, hedef kullanıcı kitlesi, teknolojik gereksinimleri ve geliştirme sürecindeki kısıtları açıklanmaktadır. Proje bir yazılım ve yapay sinir ağı projesi olduğu için gereksinimler hem uygulama davranışı hem de model eğitimi açısından değerlendirilmiştir.

2.1. Problem Tanımı

Dijital karakterlerin konuşma sırasında inandırıcı görünmesi için ağız hareketlerinin sesle uyumlu olması gerekir. Eğer çene açılması, dudak kapanması veya ağız büzülmesi konuşma sesiyle uyumsuz olursa kullanıcı karakteri doğal algılamaz. Bu durum özellikle oyunlar, sanal yayıncı sistemleri, uzaktan eğitim avatarları ve animasyon üretim araçları için önemlidir.

Manuel dudak senkronizasyonu, kare kare veya fonem bazlı düzenleme gerektirdiği için zaman alıcıdır. Donanım tabanlı yüz yakalama sistemleri ise maliyet, erişilebilirlik ve kurulum açısından her kullanıcı için uygun değildir. VocaRig bu noktada yalnızca ses girdisinden hareketle dudak/çene animasyonu üretmeyi hedefler.

Proje, bu problemi şu alt problemlere ayırır:

- Ham ses sinyalinin model için anlamlı sayısal özelliklere çevrilmesi.
- Konuşma sırasında dudak hareketlerinin zamansal süreklilikle tahmin edilmesi.
- Model çıktısının ARKit standardına uygun animasyon kanallarına dönüştürülmesi.
- Çıkarımın hem dosya tabanlı hem gerçek zamanlı akışta çalışması.
- Eğitim sürecinin izlenebilir ve tekrar üretilebilir hale getirilmesi.



Şekil 2.1. VocaRig probleminin genel giriş-çıkış ilişkisi

2.2. Hedef Kitle ve Kullanıcı Roller

VocaRig'in hedef kitlesi, konuşma tabanlı karakter animasyonu üretmek isteyen kullanıcılar ve geliştiricilerdir. Proje doğrudan son kullanıcı ürünü olarak tasarlanmamış olsa da, arayüzü sayesinde teknik olmayan kullanıcıların da deneme yapmasına izin verecek bir yapı

sunar. Bunun yanında eğitim parametreleri, veri üretimi ve model dışa aktarımı gibi özellikler daha teknik kullanıcıları hedefler.

Çizelge 2.1. Hedef kullanıcı rolleri

Kullanıcı rolü	Beklenen kullanım	Temel ihtiyaç
Model geliştirici	Veri üretimi, eğitim, metrik izleme	Tekrar üretilebilir eğitim hattı
Animasyon geliştirici	Ses dosyasıyla dudak hareketi üretme	ARKit uyumlu çıktı
Oyun geliştirici	Gerçek zamanlı avatar entegrasyonu	Düşük gecikmeli çıkarım
Test kullanıcısı	UI üzerinden model deneme	Görsel önizleme ve hata geri bildirimi

2.3. Platform ve İşletim Sistemi Gereksinimleri

Proje Python tabanlıdır ve yerel bir web arayüzü üzerinden çalışır. Mevcut proje dizininde PowerShell komutları verilmiş, çalışma ortamı Windows üzerinde kurulmuştur. Buna rağmen kullanılan temel teknolojiler çapraz platform yapıya sahiptir. Python, FastAPI, PyTorch, NumPy ve Uvicorn uygun ortam sağlandığında Linux ve macOS üzerinde de çalışabilir.

Arayüz yerel sunucuda “http://127.0.0.1:8010” adresinden açılır. Bu nedenle kullanıcının modern bir web tarayıcısına sahip olması gerekir. 3B önizleme için Three.js kullanıldığından WebGL destekli bir tarayıcı tercih edilir. Eğitim işlemi CPU üzerinde yapılabilse de pratik eğitim süresi için CUDA destekli GPU önemli avantaj sağlar.

Çizelge 2.2. Platform gereksinimleri

Gereksinim	Açıklama	Gerekçe
Python 3.14+	Proje yapılandırmasında belirtilen çalışma zamanı	Backend, eğitim ve veri işleme
Windows PowerShell	Mevcut komut örnekleri bu ortama göre yazılmıştır	Kullanıcı çalışma ortamı

Modern tarayıcı	UI ve 3B önizleme için gerekir	FastAPI statik arayüzü
CUDA destekli GPU	Zorunlu değil, fakat eğitim için yararlı	Eğitim süresini azaltır
WAV ses girdisi	Dosya tabanlı çıkarım için kullanılır	Ses özellik çıkarımı
Disk alanı	Model, metrik ve veri dosyaları için gerekir	Eğitim artefaktları

2.4. Teknoloji Gereksinimleri

Proje, farklı katmanlarda farklı teknolojiler kullanmaktadır. Python ana programlama dilidir. FastAPI HTTP ve WebSocket servislerini sağlar. PyTorch model eğitimi ve çıkarımı için kullanılır. NumPy ses işleme ve veri dizisi işlemlerinde görev alır. ONNX, eğitilen modelin farklı ortamlara taşınabilmesi için dışa aktarım formatı olarak yer alır. Three.js, web arayüzünde 3B mesh önizlemesi için kullanılır. Mermaid.js ise bu raporda sistem diyagramlarının metin tabanlı olarak sunulmasını sağlar.

Çizelge 2.3. Kullanılan teknolojiler ve görevleri

Teknoloji	Gereksinim	Açıklama	Gereke
Python	Ana uygulama, eğitim, veri işleme	Geniş yapay zeka ekosistemi	Sürüm ve paket uyumu gerekir
PyTorch	GRU model eğitimi ve çıkarımı	Esnek model geliştirme	GPU kurulumu dikkat ister
NumPy	Ses ve dizi işlemleri	Hızlı sayısal işlem	Çok büyük veride bellek yönetimi gerekir
FastAPI	REST API ve WebSocket	Tip tabanlı hızlı API geliştirme	Üretim dağıtımını ayrıca plan ister
Uvicorn	ASGI sunucusu	HTTP ve WebSocket sunumu	Yerel kullanımda port yönetimi gerekir
ONNX	Model dışa aktarımı	Platformlar arası model taşıma	Her işlem aynı destek düzeyinde olmayabilir

Three.js	3B avatar/mesh önizleme	Tarayıcıda 3B görselleştirme	WebGL desteği gerekir
YAML	Eğitim konfigürasyonu	Okunabilir ayar dosyası	Girinti hatalarına duyarlıdır

2.5. Kaynak Kısıtları ve Çözüm Yaklaşımları

Yapay sinir ağı projelerinde kaynak kısıtları genellikle veri, donanım, eğitim süresi ve değerlendirme kalitesi etrafında şekillenir. VocaRig için de benzer kısıtlar vardır. Gerçek konuşma ve yüz hareketi eşleşmeleri içeren büyük bir veri kümesi hazırlamak zor olduğundan proje sentetik veri üretim hattı içerir. Bu yaklaşım başlangıç eğitimi için yeterli ve kontrol edilebilir veri üretir; ancak gerçek konuşmacı çeşitliliğini tamamen temsil etmeyebilir.

Eğitim süresi açısından GPU kullanımı önemlidir. Mevcut nihai eğitim metriklerinde cihaz olarak cuda kullanıldığı görülmektedir. Bu, eğitim işleminin GPU üzerinde yapıldığını gösterir. Buna karşın sistem CPU modunu da destekler. UI içinde cihaz seçimi auto, cpu ve cuda seçenekleriyle yapılabilir.

Gerçek zamanlı çıkarımda en önemli kısıt gecikmedir. Sistem bu nedenle ses pencerelerini küçük parçalar halinde işler ve WebSocket destekli gerçek zamanlı iletişim kullanır. Ses seviyesi eşiği de düşük enerjili bölgelerde gereksiz ağız hareketlerini azaltmak için kullanılır.

Çizelge 2.4. Kısıtlar ve çözüm yaklaşımları

Kısıt	Etki	Projedeki çözüm
Gerçek veri azlığı	Model gerçek konuşma çeşitliliğini sınırlı öğrenebilir	Sentetik veri üretimi ve BEAT hazırlama hattı
Eğitim süresi	Parametre denemeleri yavaşlayabilir	CUDA desteği, checkpoint, erken durdurma
Gerçek zamanlı gecikme	Avatar konuşmadan geride kalabilir	Küçük ses pencereleri ve WebSocket çıkarımı
Gürültülü ses	Ağız gereksiz hareket edebilir	Volume threshold ve silence loss

Ani ağız hareketleri	Görsel doğallık bozulabilir	Max step, gate ve warmup mekanizmaları
Model taşınabilirliği	Farklı runtime ortamları sınırlanabilir	ONNX dışı aktarımı

2.6. İşlevsel ve İşlevsel Olmayan Gereksinimler

VocaRig projesinde işlevsel gereksinimler, kullanıcının sistem üzerinden doğrudan yapabildiği işlemleri tanımlar. Bunlar ses dosyası seçme, model seçme, çıkarım çalıştırma, sentetik veri üretme, eğitim başlatma, eğitim durumunu izleme ve model dışı aktarma gibi davranışlardır. İşlevsel olmayan gereksinimler ise sistemin nasıl çalışması gerektiğini açıklar. Bunlar gecikme, doğruluk, tekrar üretilebilirlik, taşınabilirlik, arayüz kullanılabilirliği ve hata dayanıklılığı gibi niteliklerdir.

Bu ayrım önemlidir çünkü yapay sinir ağı projelerinde yalnızca "model çalışıyor" demek yeterli değildir. Model çalışsa bile çıktı gecikmeli gelebilir, sessizlikte ağız hareketi üretebilir, arayüz model hatasını kullanıcıya açıklamayabilir veya eğitim çıktıları tekrar üretilmeyebilir. Bu nedenle VocaRig tasarımında hem model başarısı hem de sistem davranışı birlikte ele alınmıştır.

Çizelge 2.5. İşlevsel gereksinimler

No	Gereksinim	Açıklama	Doğrulama biçimi
F1	WAV dosyasıyla çıkarım	Kullanıcı seçili veya varsayılan WAV dosyasıyla dudak çıktısı üretebilmelidir	"/api/infer/audio" testi
F2	Gerçek zamanlı çıkarım	Mikrofon veya küçük ses parçalarıyla anlık çıktı alınabilmelidir	"/api/infer/realtime" ve WebSocket testi
F3	Model seçimi	"models" dizinindeki checkpoint dosyaları UI üzerinden seçilebilmelidir	"/api/models" ve "/api/models/select"
F4	Sentetik veri üretimi	UI veya script ile yeni .npz veri seti üretilenmelidir	sentetik veri testleri

F5	Eğitim başlatma	Kullanıcı veri seti ve parametrelerle eğitim başlatabilmelidir	training request testleri
F6	Metrik izleme	Eğitim sonuçları grafik ve JSON olarak incelenebilmelidir	“/api/metrics” testi
F7	ONNX export	Eğitilen checkpoint dış ortama aktarılabilirdir	export fonksiyonu ve training testleri
F8	ARKit çıktı	Model çıktısı 52 kanallı ARKit sözlüğüne dönüşmelidir	contract testleri

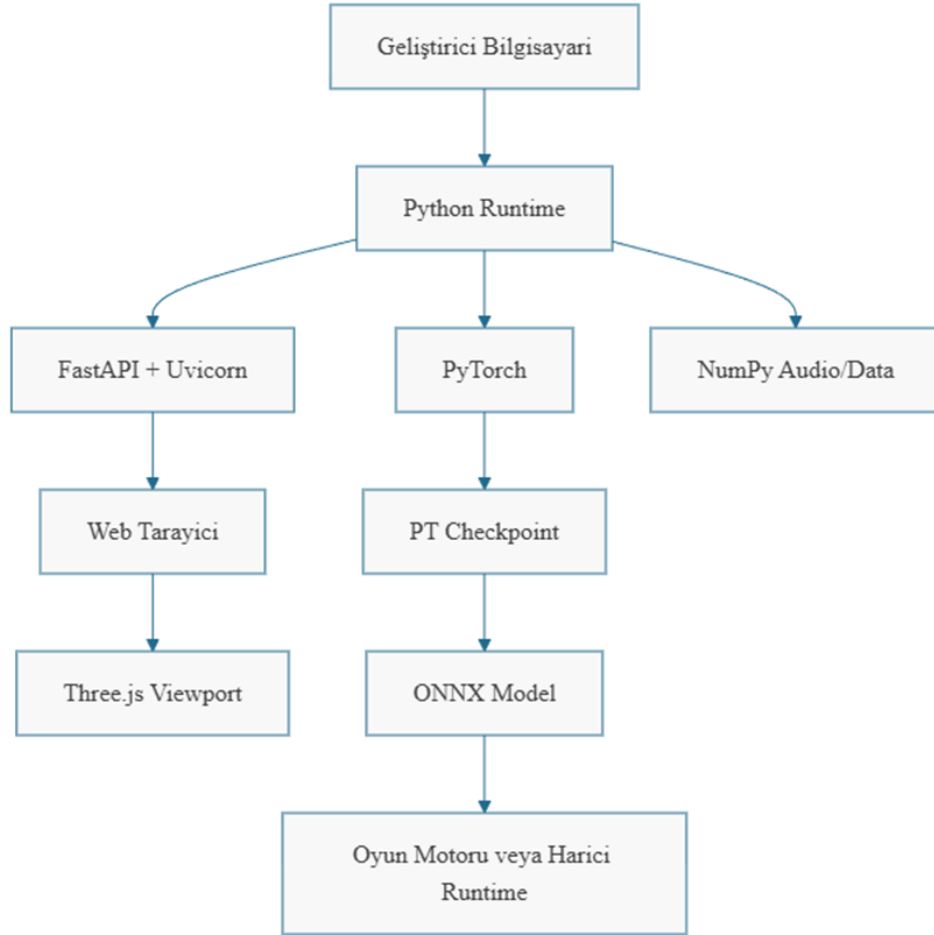
Çizelge 2.6. İşlevsel olmayan gereksinimler

Gereksinim	Beklenen nitelik	Doğrulama biçimi
Tekrar üretilebilirlik	Aynı seed ile benzer veri ve eğitim davranışı	seed: 42, deterministik split
Düşük gecikme	Canlı kullanımda hızlı model yanıtı	ModelWorker, CUDA desteği, benchmark
Veri bütünlüğü	Dataset alanları beklenen boyutta olmalı	load_npz_dataset doğrulamaları
Aralık güvenliği	Blendshape değerleri 0-1 aralığında kalmalı	clamp ve testler
Hata görünürlüğü	API hataları kullanıcıya anlamlı dönmeli	FastAPI HTTPException kullanımı
Taşınabilirlik	Model başka runtime'a aktarılabilirdi	ONNX export
Modülerlik	Ses, model, eğitim ve UI ayrılmalı	audio, models, training, ui paketleri

2.7. Sistem Bağlamı ve Dış Bağımlılıklar

VocaRig tek başına bir model dosyası değildir; çevresindeki araçlarla birlikte çalışan bir üretim hattıdır. Bu nedenle dış bağımlılıklar doğru anlaşılmalıdır. Python çalışma zamanı ve paketleri backend ve eğitim için gereklidir. Tarayıcı arayüzü statik dosyaları çalıştırır. PyTorch

eğitim ve çıkarım tarafında ana derin öğrenme altyapısıdır. ONNX dışı aktarımı, modelin proje dışındaki motorlara taşınabilmesi için kullanılır.



Şekil 2.2. Sistem bağlamı ve dış bağımlılıklar

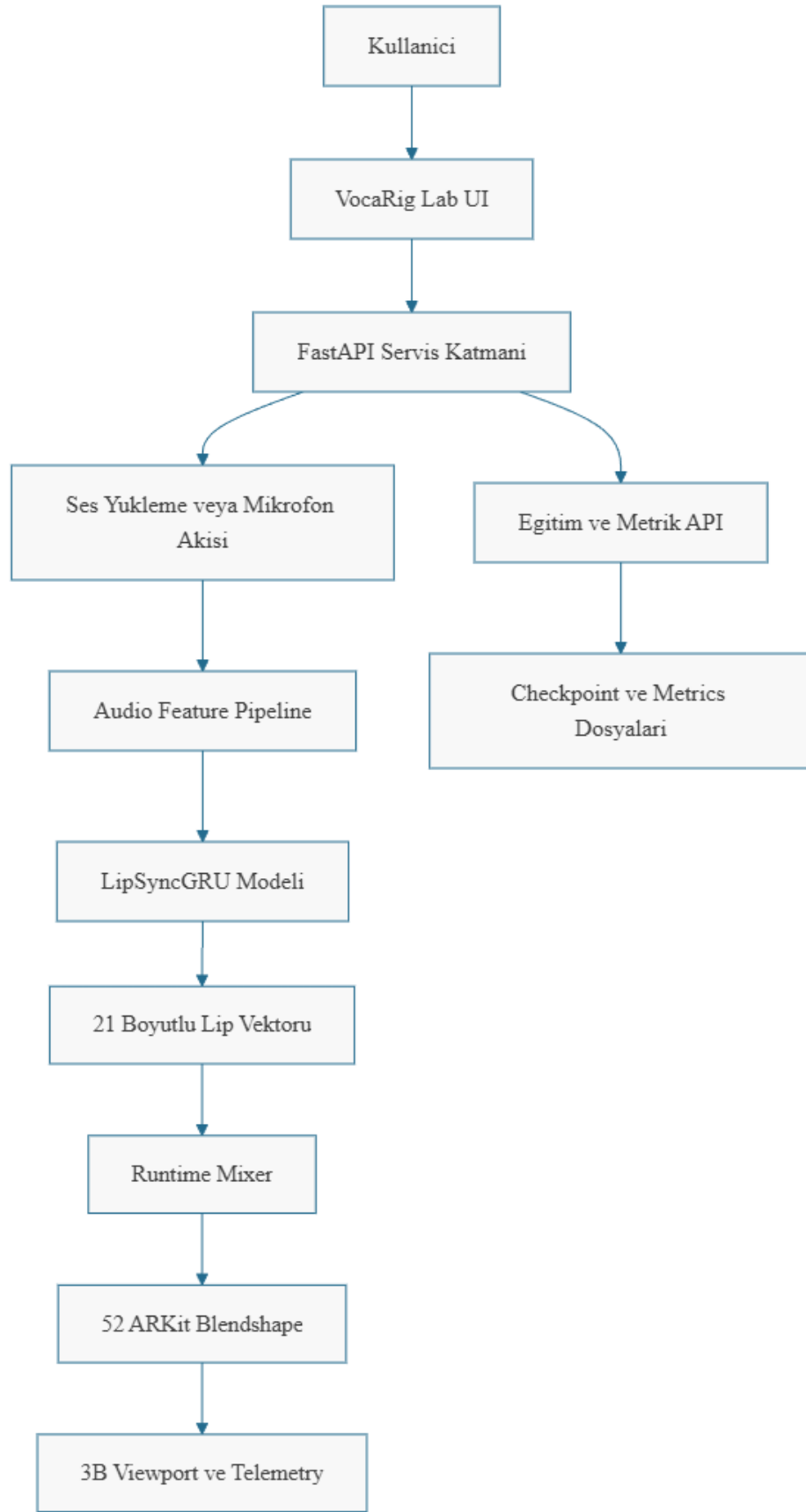
Bu bağlamda en kritik bağımlılık PyTorch ve CUDA uyumudur. CPU ile eğitim mümkün olsa da, uzun epoch sayıları ve büyük sentetik veri setleri pratik sürede sonuç almak için GPU kullanımını gerekli hale getirebilir. Arayüz tarafında ise tarayıcı ve WebGL desteği 3B önizleme için önemlidir.

3. TASARIM

Bu bölümde VocaRig sisteminin tasarımı açıklanmaktadır. Tasarım, veri hattı, model mimarisi, servis katmanı, kullanıcı arayüzü ve eğitim süreçleri üzerinden ele alınmıştır.

3.1. Genel Sistem Mimarisi

VocaRig'in genel mimarisi, ses girdisinin işlenmesiyle başlayan ve ARKit uyumlu animasyon çıktısıyla biten ardışık bir hat üzerine kuruludur. Kullanıcı WAV dosyası yükleyebilir, varsayılan ses dosyasını kullanabilir veya mikrofon üzerinden canlı ses akışı verebilir. Backend katmanı ses örneklerini uygun örnekleme hızına çevirir, log-mel özellik pencereleri oluşturur ve modeli çağırır. Model, 21 boyutlu dudak kontrol vektörü üretir. Bu vektör runtime mixer katmanında 52 boyutlu ARKit sözlüğüne yerleştirilir.



Şekil 3.1. VocaRig genel sistem mimarisi

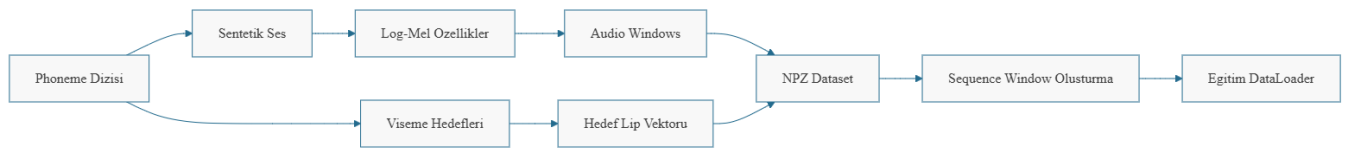
Sistemde eğitim ve çıkarım aynı proje içinde yer alır ancak görevleri ayrıdır. Eğitim hattı veri setini yükler, modeli eğitir, checkpoint ve metrik dosyaları üretir. Çıkarım hattı ise seçili checkpoint'ı yükler ve ses girdisini anlık çıktıya dönüştürür. Bu ayrım sayesinde eğitim sırasında üretilen artefaktlar UI üzerinden seçilerek denenebilir.

3.2. Veri Tasarımı

VocaRig veri tasarımında ana format “.npz” dosyasıdır. Bu dosya, modelin ihtiyaç duyduğu dizileri birlikte saklar. Veri setinde ses özellik pencereleri, önceki dudak durumu, zaman değerleri, stil değerleri ve hedef dudak vektörleri bulunur. Eğitim sırasında bu frame düzeyindeki veriler sabit uzunluklu sequence pencerelerine dönüştürülür.

Ana veri alanları şunlardır:

- audio_windows: Her kare için 11 bağlam frame'i ve 80 mel bandından oluşan ses penceresi.
- previous_lip: Önceki dudak kontrol vektörü.
- time_values: Delta time, ses güncelleme zamanı ve enerji gibi zamanla ilişkili değerler.
- style_values: Stil ve asimetri kontrol değerleri.
- y: Hedef 21 boyutlu dudak kontrol vektörü.
- utterance_ids: Cümle veya sözce bazlı ayırım.
- frame_ids: Zaman sıralaması için frame kimliği.



Şekil 3.2. Sentetik veri üretimi ve eğitim veri hattı

Bu veri tasarımı iki önemli avantaj sağlar. Birincisi, model girdileri ve hedefleri aynı dosyada taşındığı için eğitim tekrar üretilebilir olur. İkincisi, utterance kimlikleri sayesinde eğitim ve doğrulama bölünmesi rastgele frame düzeyinde değil, mümkün olduğunda sözce düzeyinde yapılır. Bu durum doğrulama setine eğitimde görülen aynı sözcenin farklı frame'lerinin sızmasını azaltır.

Çizelge 3.1. Veri alanları ve görevleri

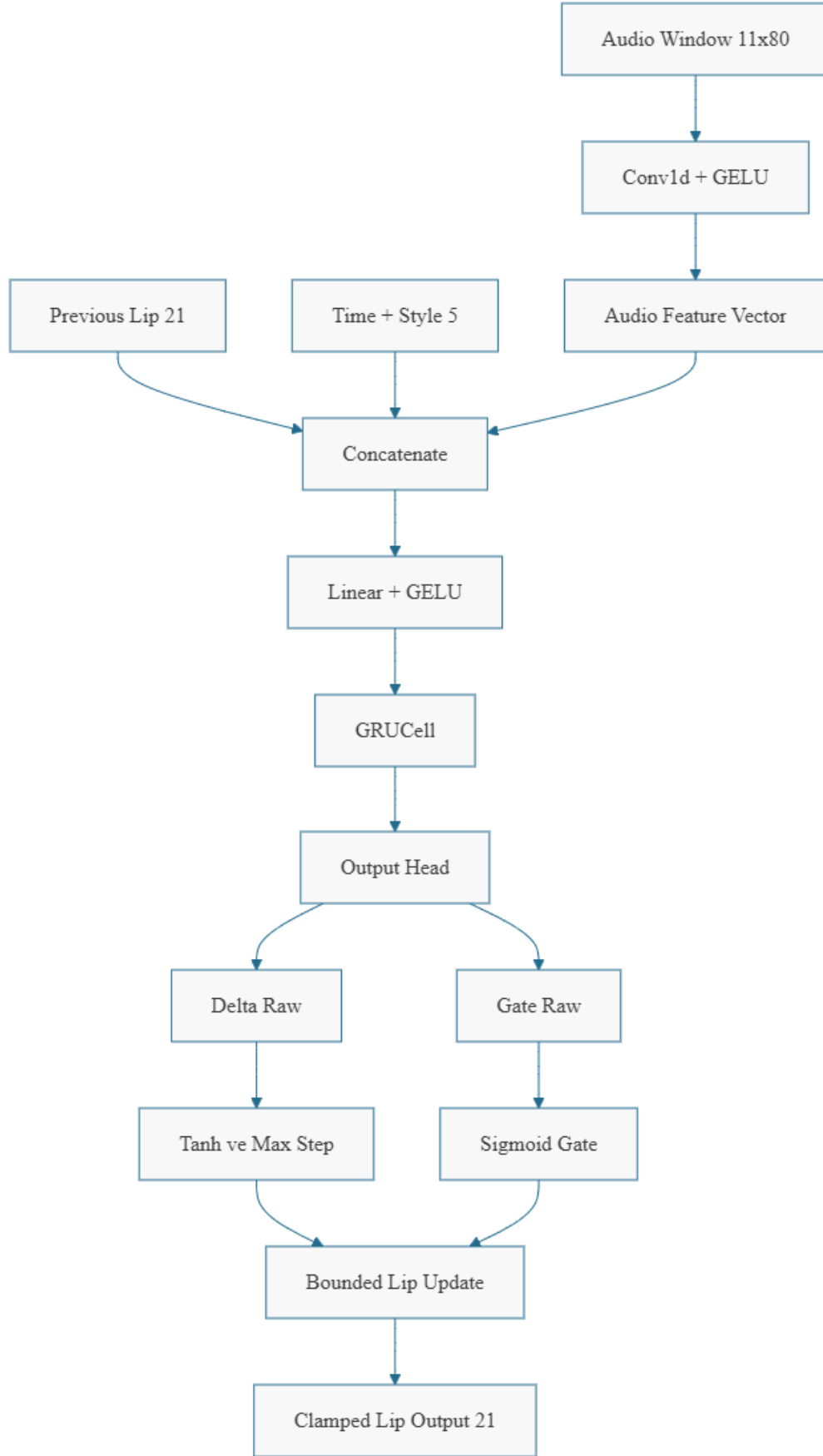
Alan	Boyut mantığı	Görev
audio_windows	$N \times 11 \times 80$	Ses bağlamını modele verir
previous_lip	$N \times 21$	Önceki ağız durumunu temsil eder
time_values	$N \times 3$	Zaman, güncelleme ve enerji bilgisi taşır
style_values	$N \times 2$	Stil ve asimetri etkisi sağlar
y	$N \times 21$	Öğrenilecek hedef dudak vektörüdür
utterance_ids	N	Grup bazlı train/validation ayrımı sağlar
frame_ids	N	Sıralı pencere üretiminde kullanılır

3.3. Model Tasarımı

Modelin merkezinde LipSyncGRU sınıfı yer alır. Model nedensel çalışacak biçimde tasarlanmıştır; yani her adımda mevcut ses bağlamı, önceki dudak durumu ve zaman/stil değerleri kullanılarak bir sonraki dudak durumu tahmin edilir. Bu yapı gerçek zamanlı kullanım için uygundur çünkü model tüm ses dosyasını bilmek zorunda değildir.

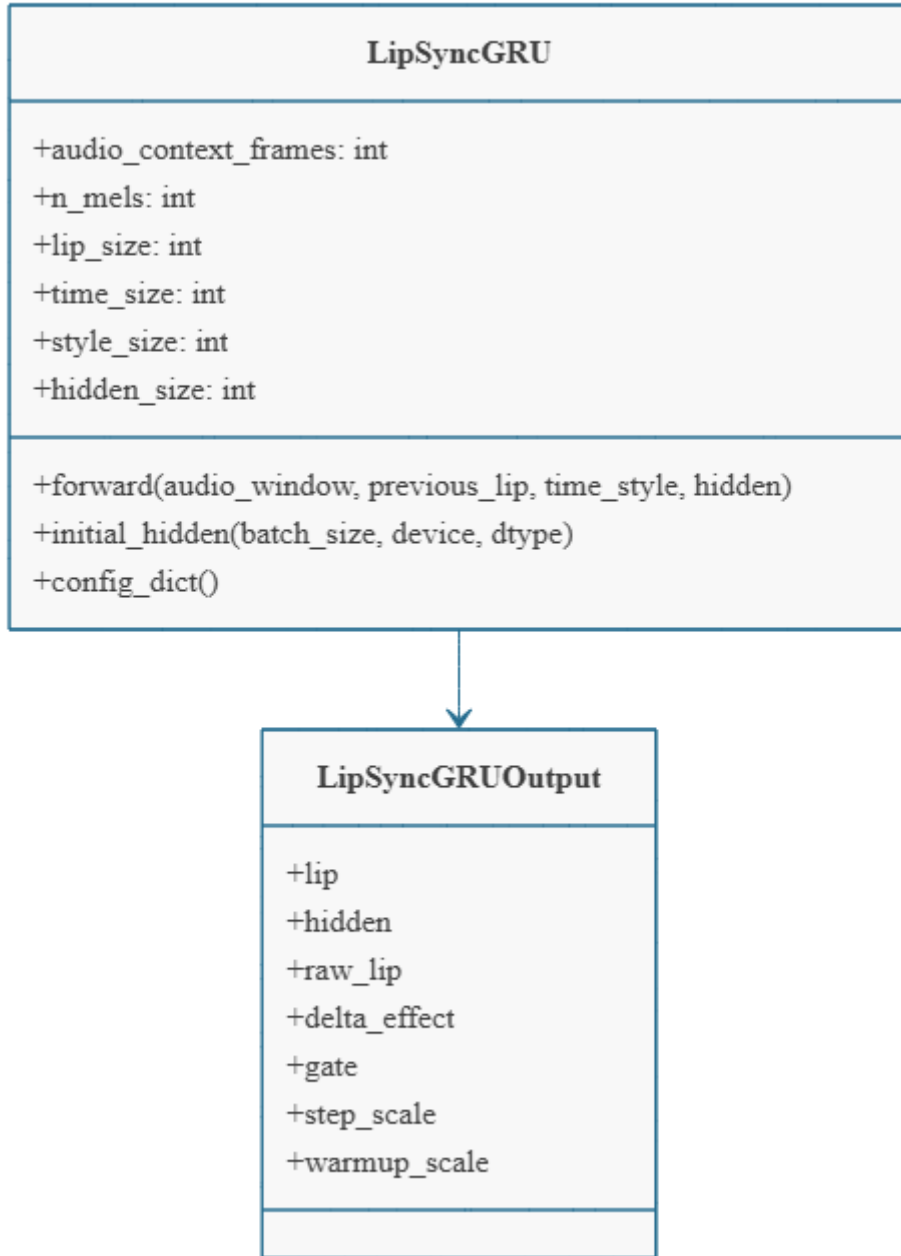
Modelin ana bileşenleri şunlardır:

- Ses kodlayıcı: 80 mel bandını Conv1d katmanlarıyla işler.
- Girdi projeksiyonu: Ses özellikleri, önceki dudak vektörü ve zaman/stil değerlerini tek temsile dönüştürür.
- GRUCell: Zamansal durumu taşır.
- Çıkış başlığı: Dudak değişimi ve kapı değerlerini üretir.
- Sınırlama katmanı: Çıktıyı 0 ile 1 aralığında tutar.



Şekil 3.3. LipSyncGRU modelinin bileşenleri

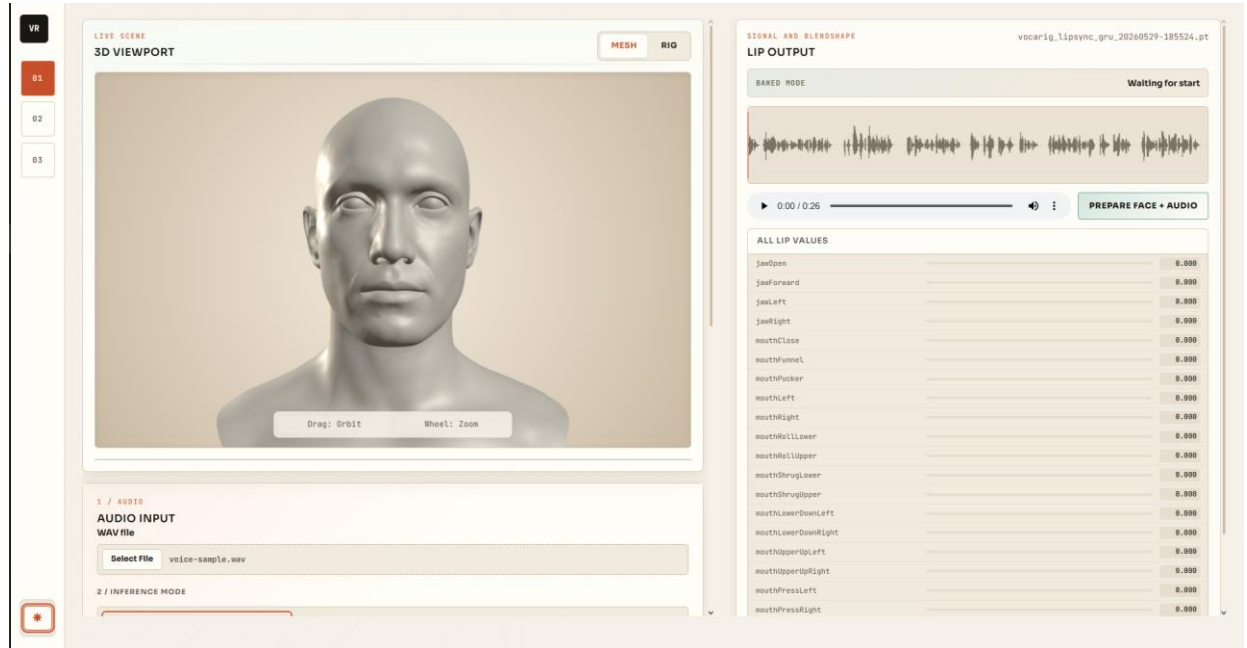
Modelin ileri geiři, konuřma animasyonu aısından nemli iki davranıřı destekler. Birincisi, max_step deęeri ile tek karedeki maksimum deęiřim sınırlanır. Bu, ağız hareketlerinde sıçrama oluřmasını engellemeye yardımcı olur. İkincisi, warmup_steps ile model durumu sıfırlandığında ilk karelerdeki geiř yumuřatılır. Bylece yeni bir akıř bařlatıldığında ağız pozunun aniden deęiřmesi azaltılır.



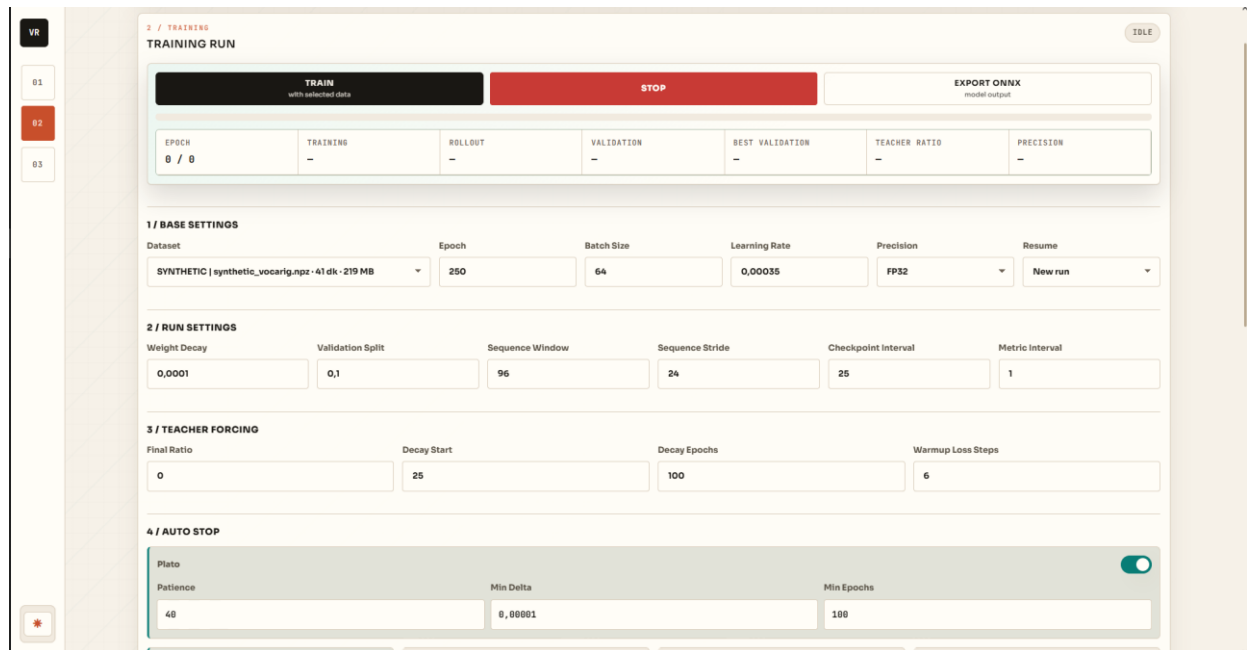
řekil 3.4. Model sınıf yapısı

3.4. Çalışma Zamanı ve Arayüz Tasarımı

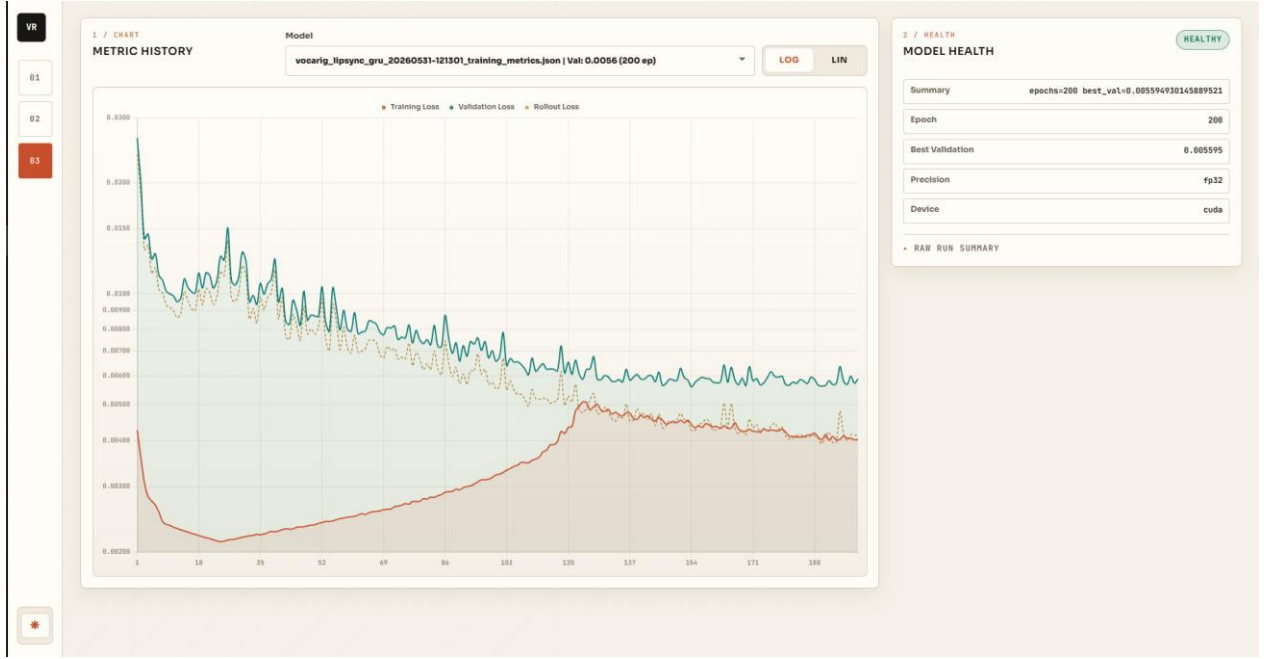
VocaRig Lab arayüzü üç ana panelden oluşur: Inference, Pipeline ve Analytics. Inference paneli ses girdisi, model seçimi, zamanlama ayarları, stil kontrolleri, dudak barları, ARKit tablo çıktısı ve 3B viewport içerir. Pipeline paneli sentetik veri üretimi, eğitim başlatma, eğitim kontrolü ve dışa aktarım işlemlerini kapsar. Analytics paneli ise eğitim metriklerini ve model değerlendirmelerini incelemek için kullanılır.



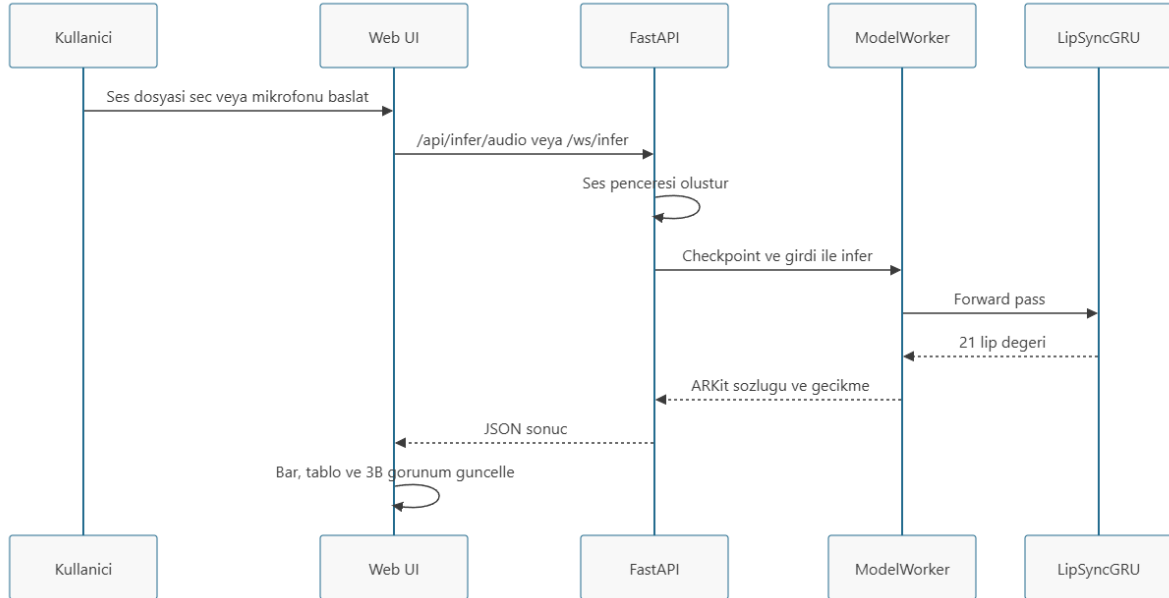
Şekil 3.5. VocaRig Lab Inference paneli ekran görüntüsü yer tutucusu



Şekil 3.6. VocaRig Lab Pipeline paneli ekran görüntüsü yer tutucusu



Şekil 3.7. VocaRig Lab Analytics paneli ekran görüntüsü yer tutucusu



Şekil 3.8. Gerçek zamanlı çıkarım sırası

Çalışma zamanında kullanıcı üç farklı çıkarım biçimi kullanabilir. Baked modda tüm WAV dosyası önce işlenir ve kareler hazırlandıktan sonra oynatılır. No-baked modda WAV oynarken küçük ses parçaları modele gönderilir. Real test modunda ise mikrofon akışı doğrudan modele bağlanır. Bu üç mod, geliştirme ve test sürecinde farklı ihtiyaçlara cevap verir.

3.5. Eğitim ve Dışa Aktarım Tasarımı

Eğitim tasarımında “TrainingRunConfig” veri sınıfı merkezi rol oynar. Eğitim konfigürasyonu YAML dosyasından, komut satırı argümanlarından veya UI isteklerinden oluşturulur. Model eğitimi sırasında checkpoint, eğitim metrikleri ve en iyi model durumu saklanır. Dışa aktarım aşamasında seçili PyTorch checkpoint dosyası ONNX formatına dönüştürülür.

Eğitim hattında öğretmen zorlaması, erken durdurma, kayıp bileşenleri, sessizlik cezası ve hareket türevi cezaları gibi ayarlar bulunur. Bu ayarlar, modelin yalnızca hedef pozlara yaklaşmasını değil, aynı zamanda zaman içinde doğal hareket üretmesini amaçlar.

3.6. API ve Veri Arayüzü Tasarımı

VocaRig servis katmanı HTTP endpointleri ve WebSocket yollarından oluşur. HTTP endpointleri çoğunlukla tekil işlem veya durum sorgusu için kullanılır. WebSocket yolları ise sürekli değişen eğitim, sentetik veri üretimi ve gerçek zamanlı çıkarım durumlarını iletmek için uygundur. Bu tasarım, UI tarafında hem klasik istek-cevap hem de canlı güncelleme gereksinimlerini karşılar.

Çizelge 3.2. Ana API yolları ve görevleri

Yol	Yöntem	Görev
/	GET	Statik VocaRig Lab arayüzünü döndürür
/api/status	GET	Blendshape adları, dosya yolları, cihaz ve model durumunu verir
/api/device	GET/POST	CPU, CUDA veya auto cihaz seçimini yönetir
/api/models	GET	Kullanılabilir checkpoint dosyalarını listeler
/api/models/select	POST	Aktif modeli seçer
/api/infer	POST	Tek adımlı model çıkarımı yapar
/api/infer/audio	POST	WAV dosyasını karelere dönüştürüp çıkarım yapar

/api/infer/realtime	POST	Küçük ses parçasıyla gerçek zamanlı çıkarım yapar
/api/synthetic/generate	POST	Sentetik veri üretimini başlatır
/api/train/start	POST	Eğitim işini başlatır
/api/train/stop	POST	Eğitim durdurma isteği gönderir
/api/export	POST	Seçili modeli ONNX formatına aktarır
/api/benchmark	POST	Model çıkarım süresini ölçer

Çizelge 3.3. WebSocket yolları

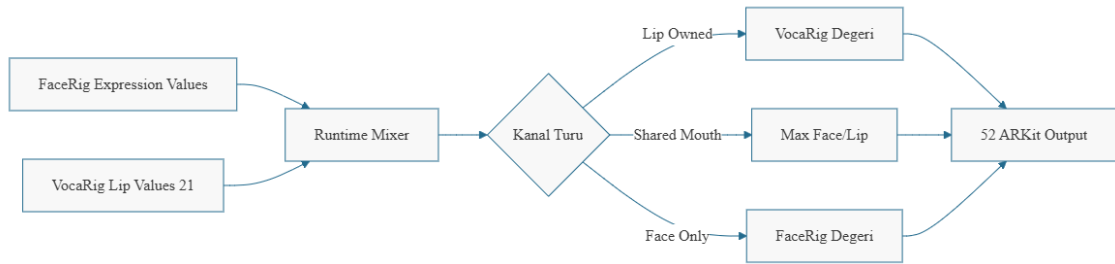
Yol	Görev	Kullanım nedeni
/ws/stream	Tek adımlı inference yanıtlarını canlı gönderir	Düşük gecikmeli UI güncellemesi
/ws/infer	Gerçek zamanlı ses çıkarımını taşır	Mikrofon ve no-baked modları
/ws/train	Eğitim durumunu periyodik gönderir	Eğitim progress takibi
/ws/synthetic	Sentetik veri üretim durumunu gönderir	Dataset üretim progress takibi

API tasarımında güvenlik açısından proje dışı model veya dataset seçimini sınırlayan kontroller bulunur. Örneğin model seçimi yalnızca proje içindeki models diziniyle sınırlandırılır. Dataset seçimi de data/synthetic veya data/processed altında olmalıdır. Bu kontroller, kullanıcının yanlışlıkla proje dışındaki dosyaları model veya veri seti gibi kullanmasını engeller.

3.7. ARKit ve FaceRig Birleşim Tasarımı

VocaRig'in blendshape tasarımı üç ayrı listeye dayanır: tam ARKit listesi, FaceRig ifade listesi ve VocaRig lip listesi. Tam ARKit listesi 52 kanaldan oluşur. VocaRig yalnızca 21 dudak/çene kanalını üretir. FaceRig tarafı ise kaş, göz, yanak ve bazı ağız ifadelerini temsil eder. Bu ayırım, modüler yüz animasyonu için önemlidir.

Runtime mixer katmanı bu iki kaynağı birleştirirken öncelik kuralları uygular. Dudak hareketinden tamamen VocaRig sorumluydu ilgili kanallar VocaRig çıktısıyla yazılır. Paylaşılan ağız kanallarında ise FaceRig ve VocaRig değerlerinin maksimumu alınır. Böylece ifade ve konuşma aynı ağız kanalında çakıştığında güçlü olan etki korunur.



Şekil 3.9. FaceRig ve VocaRig blendshape birleşim mantığı

Bu tasarım animasyon kalitesi açısından önemlidir. Eğer konuşma sistemi tüm ağız kanallarını koşulsuz ezseydi, karakterin gülme veya ağız kenarı ifadesi kaybolabilirdi. Eğer ifade sistemi konuşma kanallarını ezseydi, dudak senkronizasyonu bozulabilirdi. Bu nedenle sahiplik ve paylaşım ayırımı doğru bir mühendislik çözümüdür.

4. GELİŞTİRME

Bu bölümde projenin uygulama ayrıntıları açıklanmaktadır. Açıklamalar mevcut proje dizinindeki gerçek dosya ve modüllere dayanmaktadır.

4.1. Proje Dizin Yapısı

VocaRig dizininde kaynak kodlar src/vocarig altında toplanmıştır. Testler tests klasöründedir. Komut satırı çalıştırıcıları scripts altında yer alır. Eğitim konfigürasyonu configs/train_config.yaml dosyasında tutulur. Eğitilmiş modeller ve metrikler models/trained altında saklanır. Varsayılan mesh ve ses dosyaları data/mesh ve data/audio dizinlerindedir.

Çizelge 4.1. Ana dizinler ve görevleri

Dizin/Dosya	Görev
src/vocarig/audio	Ses yükleme, yeniden örnekleme ve log-mel özellik çıkarımı
src/vocarig/models	Blendshape sözleşmeleri ve LipSyncGRU modeli
src/vocarig/synthetic	Sentetik konuşma ve dudak verisi üretimi
src/vocarig/training	Eğitim, veri yükleme, hassasiyet ve ONNX export
src/vocarig/runtime	Dudak vektörü ile ARKit sözlüğü birleştirme
src/vocarig/ui	FastAPI uygulaması, model worker ve statik arayüz
tests	Birim, sözleşme, eğitim ve API testleri
configs/train_config.yaml	Model, veri, eğitim ve export ayarları
models/trained	Eğitilmiş checkpoint, ONNX ve metrics dosyaları

4.2. Ses Özellik Çıkarımı

Ses işleme katmanı src/vocarig/audio/features.py dosyasında geliştirilmiştir. Bu katman WAV dosyasını mono float32 örnek dizisine dönüştürür, gerekirse örnekleme hızını doğrusal interpolasyon ile değiştirir ve log-mel spektrogram üretir. Projede varsayılan ses örnekleme hızı 16000 Hz, mel bandı sayısı 80, pencere uzunluğu 25 ms, hop uzunluğu 10 ms ve animasyon FPS değeri 30'dur.

Ses özellik çıkarım sürecinde her animasyon karesi için 11 mel bağlam karesi tutulur. Bu yaklaşım modelin yalnızca tek bir anlık ses değerine değil, kısa süreli geçmiş bağlama bakmasını sağlar. Bu bağlam konuşma seslerinin geçişlerini yakalamak için önemlidir.

Kod mantığı özetle şöyledir:

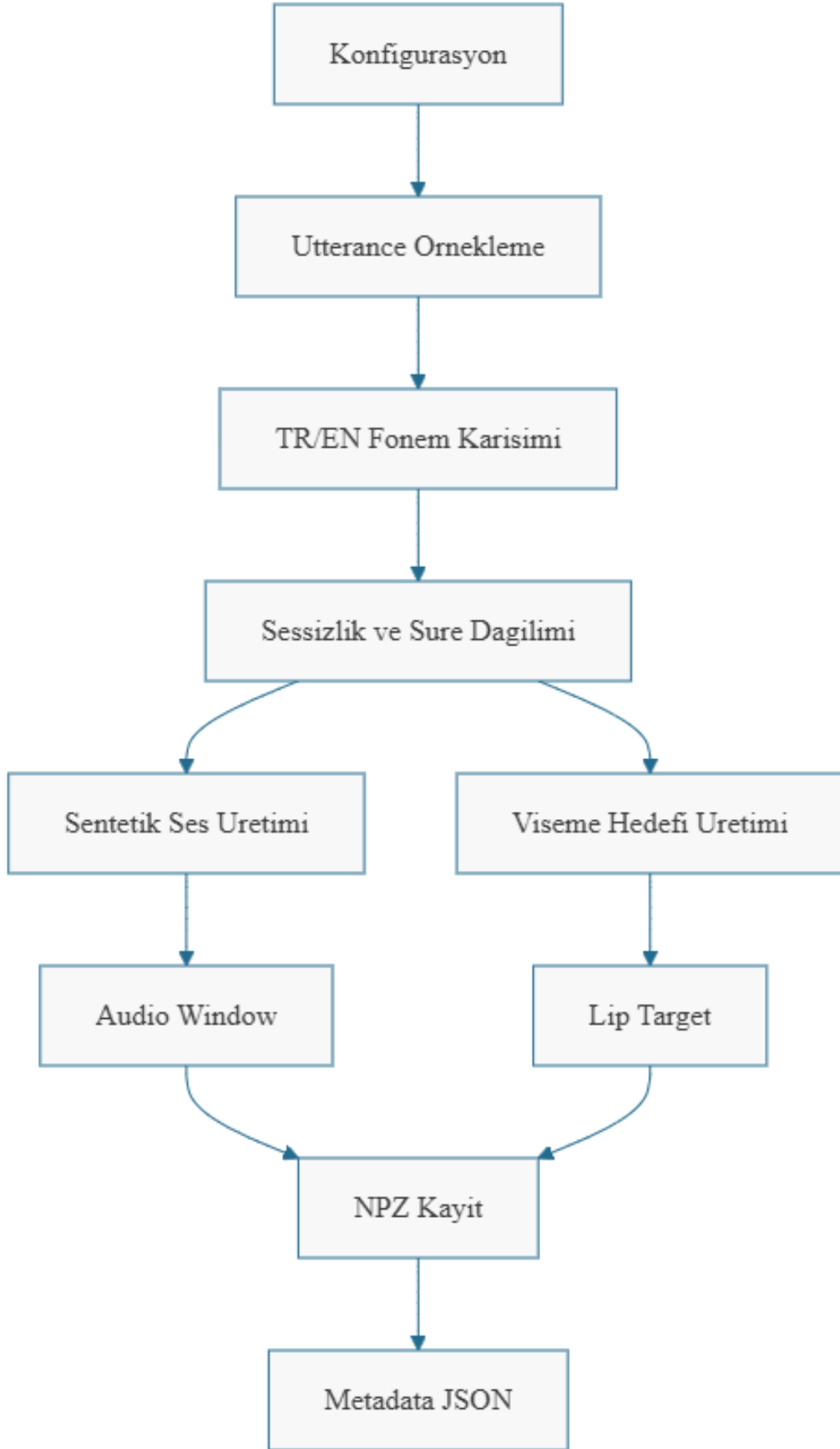
```
windows, times, energies = audio_to_feature_windows(  
    samples,  
    sample_rate,  
    audio_config,  
    volume_threshold  
)
```

Burada “windows” modele verilecek ses pencerelerini, “times” her karenin zaman bilgisini, “energies” ise ses enerjisini temsil eder. Eğer RMS enerji eşik değerinin altındaysa ilgili pencere sıfırlanır. Böylece sessiz bölgelerde modelin ağız hareketi üretmesi engellenmeye çalışılır.

4.3. Sentetik Veri Üretimi

Sentetik veri üretimi src/vocarig/synthetic modülleriyle gerçekleştirilir. Bu sistem, fonem dizileri, viseme hedefleri, sentetik ses özellikleri ve dudak hedefleri üretir. Konfigürasyonda 4000 utterance, 18-56 fonem aralığı, 0.18 sessizlik olasılığı ve Türkçe/İngilizce karışımı için 0.55/0.45 oranı tanımlanmıştır.

Sentetik veri üretimi, gerçek veri toplama maliyetini azaltır. Ayrıca fonem dağılımı, sessizlik oranı ve stil değerleri kontrollü biçimde ayarlanabilir. Bununla birlikte sentetik verinin gerçek konuşma çeşitliliğini tam olarak temsil edemeyeceği unutulmamalıdır. Bu nedenle proje içinde BEAT tabanlı veri hazırlama script'i de bulunur.



Şekil 4.1. Sentetik veri üretim akışı

4.4. Model Eğitimi

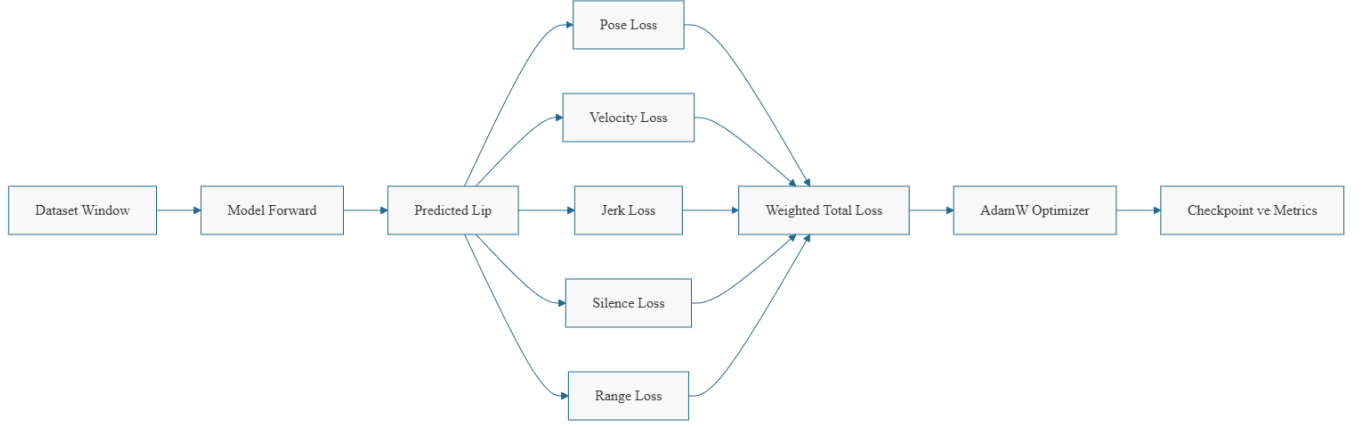
Model eğitimi “src/vocarig/training/train.py” dosyasında yürütülür. Eğitim yapılandırması “TrainingRunConfig” sınıfıyla temsil edilir. Bu sınıfta veri yolu, checkpoint yolu, metrik yolu, model boyutları, batch size, epoch sayısı, öğrenme oranı, validation split, precision, teacher forcing ve loss ağırlıkları gibi birçok parametre bulunur.

Mevcut konfigürasyonda eğitim için öne çıkan değerler şunlardır:

Çizelge 4.2. Eğitim parametreleri

Parametre	Değer
Model tipi	lipsync_gru
Audio context frames	11
Mel bandı	80
Lip output size	21
Hidden size	128
Batch size	64
Sequence window	96
Sequence stride	24
Learning rate	0.00035
Weight decay	0.0001
Validation split	0.1
Precision	fp32
Device	auto / UI seçimine göre cuda veya cpu
Loss bileşenleri	pose, velocity, jerk, silence, range

Eğitimde kayıp fonksiyonu tek bir MSE değerinden ibaret değildir. Pose loss hedef dudak pozuna yaklaşmayı sağlar. Velocity ve jerk loss ağız hareketinin zamansal pürüzsüzlüğünü destekler. Silence loss sessiz bölgelerde ağız hareketini cezalandırır. Range loss ise değerlerin beklenen aralıkta kalmasını teşvik eder.



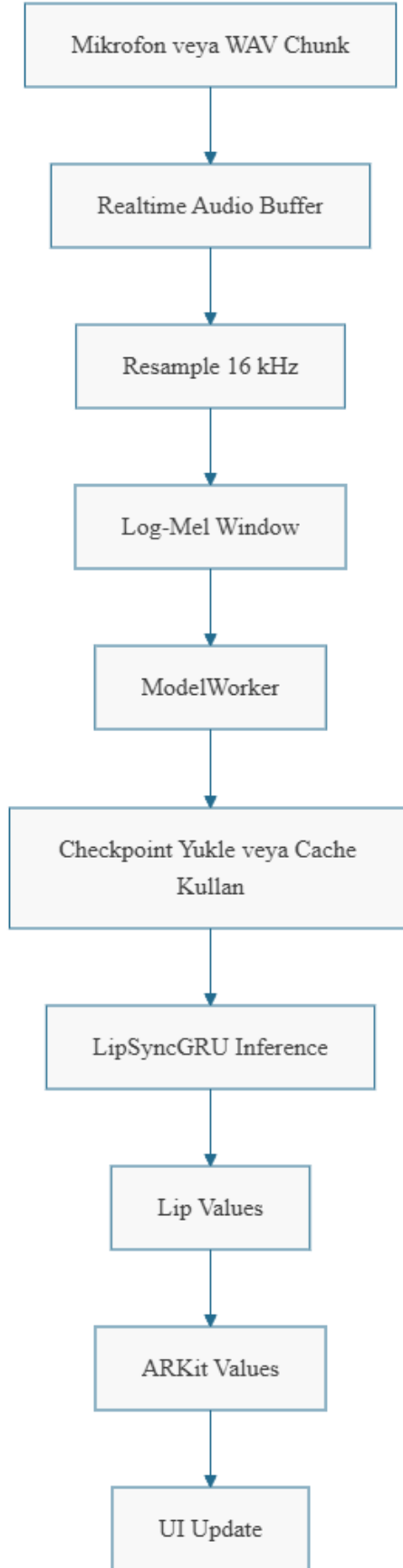
Şekil 4.2. Eğitim kayıp bileşenleri

Öğretmen zorlaması oranı epoch ilerledikçe azaltılır. Bu strateji, modelin ilk aşamada doğru hedeflere daha kolay yaklaşmasını, sonraki aşamada ise kendi önceki tahminleriyle ilerlemeyi öğrenmesini sağlar. Bu davranış gerçek zamanlı kullanım için önemlidir çünkü canlı çıkarım sırasında model sürekli kendi önceki çıktılarıyla beslenir.

4.5. Gerçek Zamanlı Çıkarım

Gerçek zamanlı çıkarım, `src/vocarig/ui/app.py`, `src/vocarig/ui/inference.py` ve `src/vocarig/ui/model_worker.py` modülleriyle gerçekleştirilir. API tarafında `/api/infer`, `/api/infer/audio`, `/api/infer/realtime`, `/ws/stream` ve `/ws/infer` yolları bulunur. Bu yollar dosya tabanlı ve canlı çıkarım biçimlerini destekler.

Gerçek zamanlı akışta ses örnekleri kısa buffer içinde tutulur. Sistem bu buffer'dan son ses penceresini üretir, enerji değerini hesaplar ve modeli çağırır. Model worker checkpoint yükleme, cihaz seçimi ve gecikme ölçümünü yönetir. Çıktı JSON olarak UI tarafına döner.



Şekil 4.3. Gerçek zamanlı çıkarım veri akışı

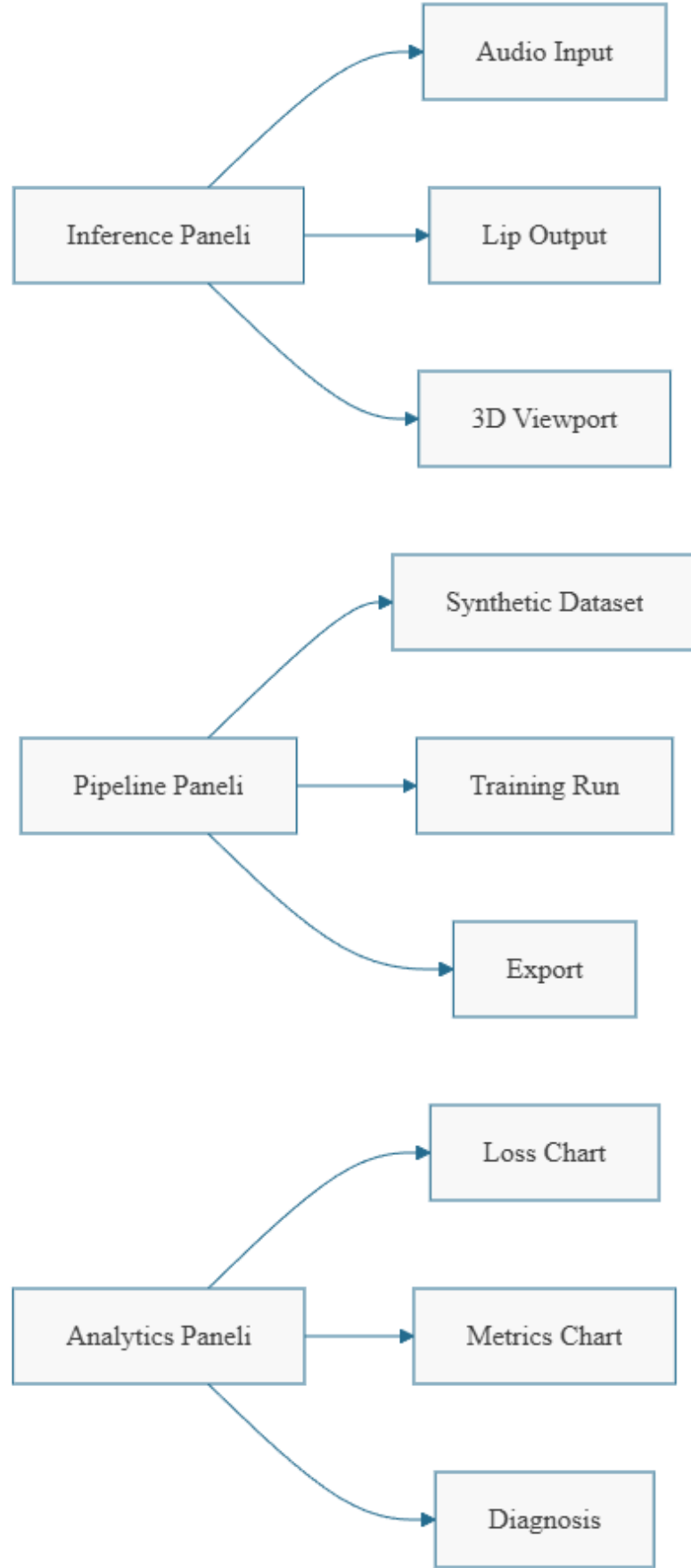
Bu yapıda gecikme ölçümü önemlidir. API yanıtlarında latency_ms veya infer_ms gibi değerler döndürülür. Böylece kullanıcı modelin CPU veya CUDA üzerinde ne kadar hızlı çalıştığını gözlemleyebilir.

4.6. Web Arayüzü

Web arayüzü src/vocarig/ui/static altında HTML, CSS ve JavaScript dosyalarından oluşur. Arayüzde sekmeli yapı kullanılmıştır. Inference sekmesi doğrudan animasyon üretimine, Pipeline sekmesi veri ve eğitim süreçlerine, Analytics sekmesi ise metrik incelemeye odaklanır.

Arayüzün önemli özellikleri şunlardır:

- Varsayılan voice-sample.wav ses dosyası.
- Model seçme listesi.
- CPU/CUDA/auto cihaz seçimi.
- Baked, no-baked ve real test modları.
- Dudak kontrol değerlerini bar olarak gösterme.
- Tam ARKit frame tablosu.
- 3B mesh veya rig önizleme.
- Sentetik veri üretimi.
- Eğitim başlatma/durdurma/temizleme.
- Metrik grafikleri.
- ONNX export ve benchmark.



Şekil 4.4. VocaRig Lab arayüz panel yapısı

4.7. Kurulum ve Çalıştırma Yönergeleri

Proje çalışma alanında sanal ortam ve script komutları hazırdır. README dosyasında temel komutlar PowerShell biçiminde verilmiştir.

Önce sentetik veri üretimi yapılır:

```
.\.venv\Scripts\python.exe scripts\generate_synthetic_data.py
```

Model eğitimi başlatılır:

```
.\.venv\Scripts\python.exe scripts\train_model.py
```

Eğitilen model ONNX formatına aktarılır:

```
.\.venv\Scripts\python.exe scripts\export_onnx.py
```

Arayüz çalıştırılır:

```
.\.venv\Scripts\python.exe scripts\run_ui.py
```

Arayüz açıldıktan sonra tarayıcıdan şu adrese gidilir:

<http://127.0.0.1:8010>

Kurulum sırasında dikkat edilmesi gereken en önemli nokta, Python sürümü ve paket bağımlılıklarının proje yapılandırmasıyla uyumlu olmasıdır. GPU ile eğitim yapılacaksa PyTorch CUDA kurulumu ayrıca doğrulanmalıdır.

4.8. Eğitim Artefaktları ve Dosya Yönetimi

Eğitim tamamlandığında yalnızca checkpoint dosyası üretilmez. VocaRig eğitim hattı model checkpoint'i, eğitim metrikleri ve uygun durumda ONNX dosyası gibi birden fazla artefakt üretir. Bu artefaktların birlikte saklanması önemlidir çünkü modelin hangi ayarlarla eğitildiği, hangi veriyle üretildiği ve hangi kayıp değerlerine ulaştığı ancak metrik dosyasıyla anlaşılır.

Mevcut proje içinde models/trained altında birden fazla eğitim çıktısı bulunmaktadır. Dosya adlarında tarih-saat bilgisi yer aldığı için farklı eğitim denemeleri birbirinden ayrılabilir. Bu yaklaşım deney takibi için yararlıdır. Kullanıcı UI üzerinden aktif modeli seçtiğinde sistem ilgili checkpoint'i kullanır, metrik seçimi yapılırken de aynı isim köküne sahip metrics dosyası incelenebilir.

Çizelge 4.3. Eğitim artefaktları

Artefakt	Uzantı	Amaç
Checkpoint	.pt	PyTorch model ağırlıkları ve konfigürasyon bilgisi
Metrics	.json	Eğitim geçmişi, kayıplar, cihaz ve run bilgileri
ONNX model	.onnx	Harici runtime veya oyun motoru entegrasyonu
Training checkpoint	.pt	Ara epoch kayıtları ve resume ihtimali
Metadata	.json	Dataset üretim veya hazırlama bilgileri

Artefakt yönetiminde tarihli dosya adları kullanılması, önceki denemelerin üzerine yazılmasını engeller. Bu, akademik raporlama için de önemlidir; çünkü nihai modelin hangi dosyadan geldiği açıkça belirtilebilir.

4.9. Geliştirme Süreci ve Git Geçmişi

Proje ekibindeki isimler git versiyon kontrolünde de görülmektedir. Git geçmişi incelendiğinde katkılarının farklı konu başlıklarına ayrıldığı anlaşılır. UI layout, default media, training optimization, artifact registry, model artifacts, test foundation, validation suite ve final training gibi işler ayrı commit ve merge başlıklarıyla ilerlemiştir. Bu durum ekip çalışmasının tek seferlik büyük bir yükleme yerine aşamalı geliştirme şeklinde yürütüldüğünü gösterir.

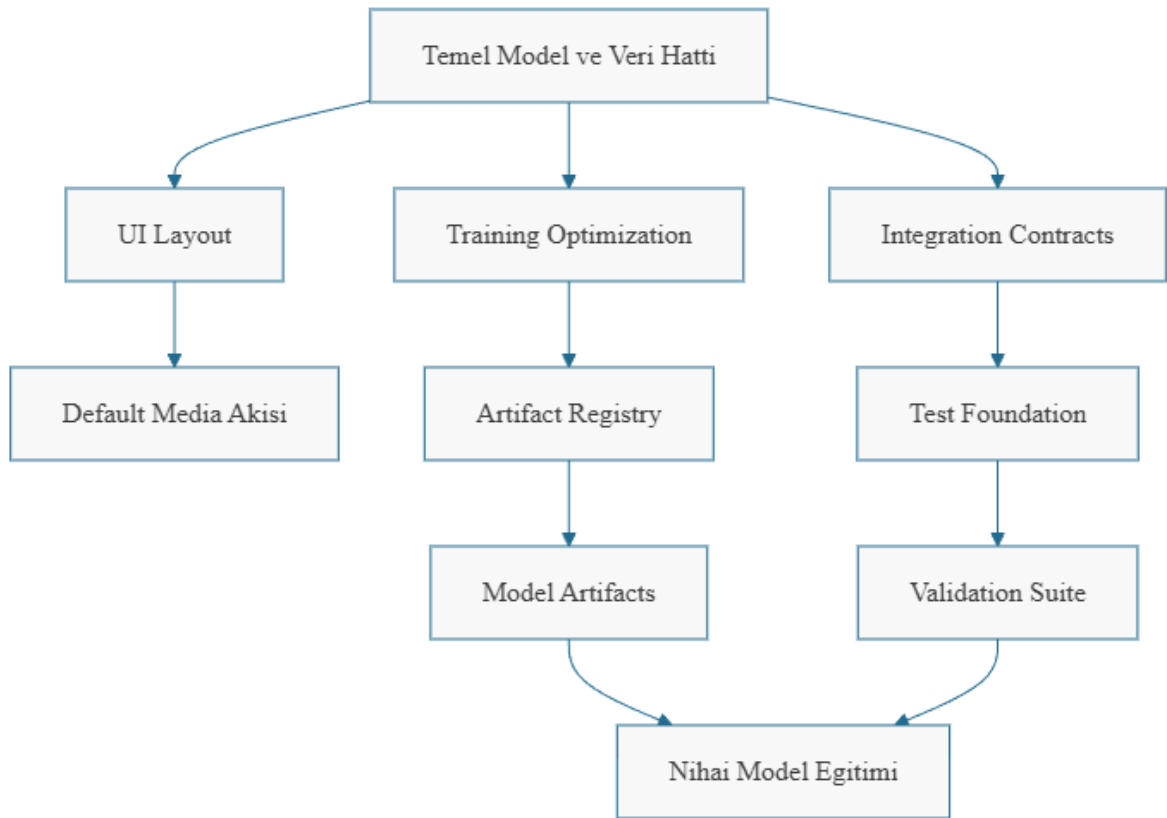
Çizelge 4.4. Git geçmişinden görülen geliştirme başlıkları

Geliştirme başlığı	İçerik
UI layout	Inference, Pipeline ve Analytics arayüz iskeleti
Default media	Varsayılan mesh ve ses dosyası akışı
Integration contracts	Blendshape ve entegrasyon sözleşmeleri
Training optimization	Eğitim konfigürasyonu ve data loader iyileştirmeleri
Artifact registry	Checkpoint ve metrik dosya yönetimi
Model artifacts	Eğitilmiş model ve metrik çıktıları
Test foundation	Contract ve synthetic data testleri
Validation suite	Training, UI ve silence cleanup testleri
Final training	Nihai eğitim çıktısı

Bu geliştirme biçimi, yazılım mühendisliği açısından olumlu bir yapıdır. Çünkü her iş parçası ayrı doğrulanabilir ve hata olduğunda ilgili değişiklik daha kolay bulunabilir.

Projenin git commit geçmişinden de görülebileceği üzere proje geliştirme süreci ekip üyeleri arasında iş bölümü yapılarak yürütülmüştür. Arayüz geliştirme, veri hazırlama, eğitim hattı, model çıktıları, test altyapısı ve doğrulama süreçleri farklı katkılarla ilerletilmiştir.

Bunun yanında model mimarisi tasarımı yalnızca tek bir ekip üyesinin çalışması olarak ele alınmamış; modelin giriş özellikleri, çıktı blendshape kanalları, eğitim yaklaşımı ve sistem içindeki kullanım biçimi ekip üyelerinin ortak değerlendirmeleriyle şekillendirilmiştir. Bu nedenle VocaRig projesinde hem yazılım geliştirme süreci hem de model mimarisi tasarımı kolektif bir ekip çalışması olarak gerçekleştirilmiştir.



Şekil 4.5. Git geçmişinden türetilen geliştirme akışı

4.10. Önemli Kod Davranışları

VocaRig kodunda dikkat çeken davranışlardan biri, veri doğrulamasının erken yapılmasıdır. Dataset yüklenirken gerekli alanlar yoksa hata üretilir. Boyutlar beklenen değerlerle uyuşmuyorsa eğitim başlamadan önce süreç durur. Bu yaklaşım, eğitim sırasında daha zor anlaşılacak hataların erken yakalanmasını sağlar.

İkinci önemli davranış, model çıktılarının sürekli 0 ile 1 aralığında tutulmasıdır. ARKit blendshape değerleri bu aralıkta yorumlandığı için modelin taşma üretmesi animasyon tarafında sorun yaratabilir. Hem model forward fonksiyonunda clamp kullanılması hem de runtime mixer içinde değerlerin sınırlandırılması bu nedenle önemlidir.

Üçüncü önemli davranış, gerçek zamanlı akışta stream kimliği kullanılmasıdır. Farklı çıkarım oturumlarının durumları birbirine karışmamalıdır. stream_id ile buffer ve model durumu ayrıştırılabilir. Bu yapı ileride çoklu kullanıcı veya çoklu avatar desteği için de genişletilebilir.

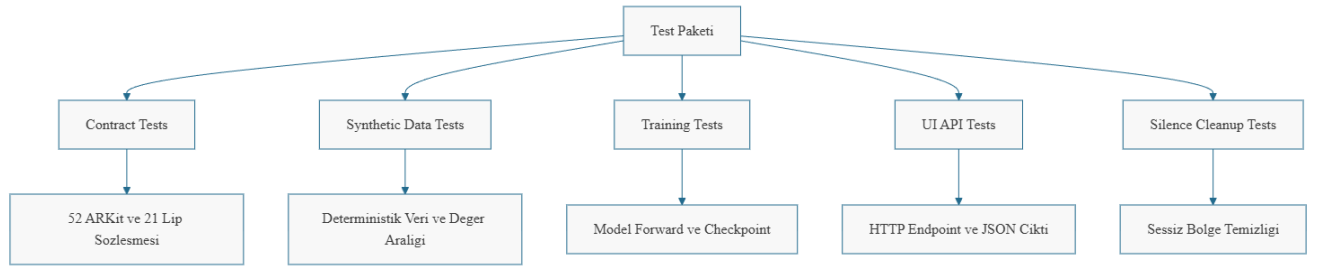
5. TEST ve DOĞRULAMA

Bu bölümde VocaRig projesinin test yaklaşımı, mevcut test dosyaları ve eğitim sonuçları açıklanmaktadır. Proje yalnızca görsel olarak çalışan bir arayüz üretmekle kalmamış, temel veri ve model sözleşmelerini testlerle doğrulamıştır.

5.1. Test Stratejisi

Test stratejisi beş ana başlıkta ele alınabilir:

- ARKit ve lip blendshape sözleşmelerinin doğrulanması.
- Sentetik veri üretiminin şekil, aralık ve deterministiklik açısından test edilmesi.
- Model ileri geçişi, teacher forcing ve eğitim çıktı dosyalarının doğrulanması.
- UI/API endpointlerinin beklenen JSON yapılarıyla çalışmasının kontrol edilmesi.
- Sessizlik temizleme ve düşük enerji davranışının test edilmesi.



Şekil 5.1. Test kapsamı ilişkisi

5.2. Birim ve Sözleşme Testleri

tests/test_contracts.py dosyası ARKit ve lip blendshape listelerinin beklenen uzunlukta olduğunu doğrular. ARKit listesi 52 kanaldan, VocaRig lip listesi 21 kanaldan oluşur. Bu test önemlidir çünkü model çıktısı, runtime mixer ve UI tablo yapısı bu sabit sözleşmeye dayanır.

tests/test_synthetic_generation.py sentetik veri üretiminin tekrar üretilebilir olduğunu ve değerlerin 0 ile 1 aralığında kaldığını doğrular. Ayrıca sessiz viseme hedeflerinde jawOpen değerinin düşük kalması gibi davranışları test eder.

tests/test_training.py modelin çıktı boyutlarını, değer aralığını, maksimum değişim sınırını, teacher forcing oranını, grouped split davranışını, kısa sequence padding davranışını ve eğitim sonucunda checkpoint/metrik/ONNX artefaktlarının oluşmasını kontrol eder.

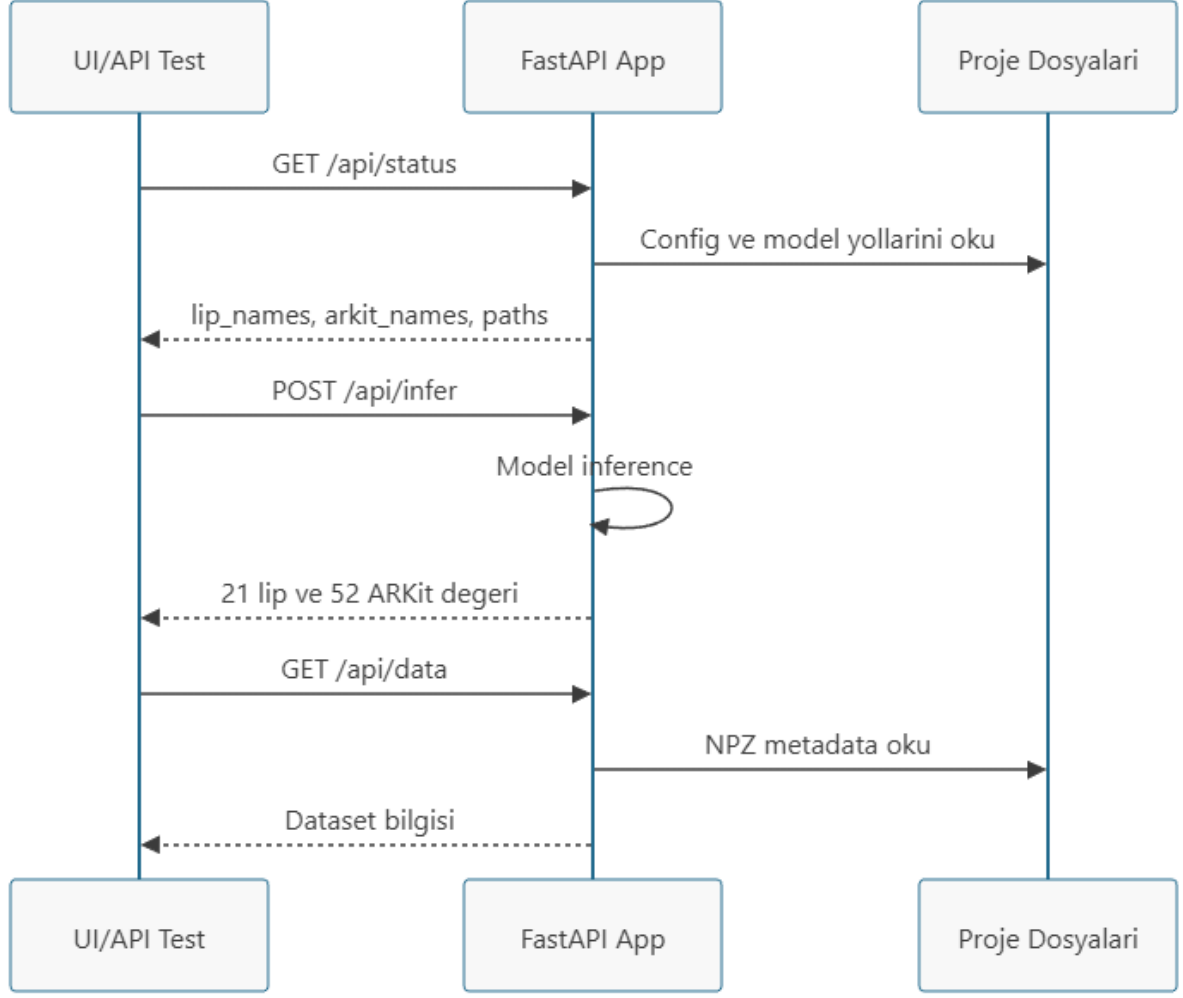
Çizelge 5.1. Test dosyaları ve doğruladıkları davranışlar

Test dosyası	Doğrulanın davranış
test_contracts.py	ARKit ve lip blendshape sözleşmeleri, FaceRig/VocaRig merge davranış
test_synthetic_generation.py	Sentetik veri şekli, deterministik üretim, değer aralığı
test_training.py	Model forward, teacher forcing, dataset split, eğitim artefaktları
test_ui_api.py	UI ana sayfası, API status, inference, realtime, data, metrics, training request
test_beat_silence_cleanup.py	Düşük enerjili bölgelerin sıfırlanması ve previous_lip sürekliliği

5.3. UI/API Doğrulaması

UI/API testleri, FastAPI uygulamasının beklenen endpointleri sağladığını doğrular. /api/status yanıtında 21 lip adı ve 52 ARKit adı dönmesi beklenir. /api/infer ve /api/infer/realtime yolları model çıktılarının beklenen uzunlukta olup olmadığını kontrol eder. /api/data, /api/metrics, /api/datasets gibi endpointler veri ve metrik dosyalarının UI tarafından okunabilir olmasını sağlar.

Arayüzde varsayılan medya dosyaları da önemlidir. Projede mesh dosyası data/mesh/ARKitMesh.glb, ses dosyası ise data/audio/voice-sample.wav altında tutulur. FastAPI uygulaması bu dizinleri /mesh ve /audio statik yolları olarak mount eder. Böylece UI içinde varsayılan model ve ses dosyası doğrudan erişilebilir olur.



Şekil 5.2. UI/API doğrulama akışı

5.4. Eğitim Sonuçları

Mevcut proje dizinindeki en güncel nihai eğitim metrik dosyası models/trained/vocarig_lipsync_gru_20260531-121301_training_metrics.json olarak görülmektedir. Bu dosyada tamamlanan epoch sayısı, en iyi doğrulama kaybı, son eğitim kaybı, son doğrulama kaybı, hassasiyet ve cihaz bilgileri yer almaktadır.

Çizelge 5.2. Nihai eğitim metrikleri

Metrik	Değer
Run ID	run_20260531-121305_3ef7f683ce
Completed epochs	200
Best validation loss	0.00559493014588952
Final train loss	0.00403076689690351
Final validation loss	0.00588653050363064

Precision	fp32
AMP enabled	False
Device	cuda
Early stopped	False
Checkpoint	models/trained/vocarig_lipsync_gru_20260531-121301.pt

[NİHAİ MODEL EĞİTİM GRAFİĞİ buraya eklenecek]

Şekil 5.3. Nihai model eğitim grafiği yer tutucusu

Bu sonuçlarda son eğitim kaybının son doğrulama kaybından düşük olması beklenen bir durumdur. Eğitim verisi doğrudan optimize edildiği için model eğitim kümesinde daha düşük hata yapar. Doğrulama kaybı ise görülmeyen veri üzerindeki genelleme davranışını gösterir. En iyi doğrulama kaybı ile son doğrulama kaybı arasında büyük bir kopma olmaması, eğitim sonunda ciddi bir bozulma olmadığını gösterir.

Bu Nihai modelin metrikleri, modelin yalnızca eğitim verisine uyum sağlamadığını, aynı zamanda ayrılmış doğrulama verisi üzerinde de kararlı davrandığını göstermektedir. Eğitim süreci 200 epoch boyunca CUDA cihazı üzerinde tamamlanmıştır. Eğitim sonunda final train loss 0.00403, final validation loss 0.00589 ve final rollout loss 0.00415 olarak ölçülmüştür. En iyi doğrulama kaybı ise 154. epochta 0.00559 değeriyle elde edilmiştir.

Train loss değerinin validation loss değerinden düşük olması beklenen bir durumdur. Çünkü model eğitim verisi üzerinde doğrudan optimize edilir. Burada önemli olan nokta, final validation loss değerinin best validation loss değerinden çok uzaklaşmamasıdır. Best validation loss 0.00559 iken final validation loss 0.00589 seviyesinde kalmıştır. Bu fark küçük olduğu için eğitimin sonunda belirgin bir overfitting bozulması görülmemektedir.

Rollout loss değeri ayrıca önemlidir. Normal eğitim kaybında model bazı aşamalarda hedef önceki dudak değerlerinden destek alabilir. Rollout değerlendirmesi ise modelin kendi ürettiği önceki çıktılarla ilerlediği duruma daha yakındır. Bu nedenle rollout loss, canlı kullanım ve ardışık çıkarım davranışı hakkında daha gerçekçi sinyal verir. Final rollout loss değerinin 0.00415 olması, modelin serbest akışta da kararlı dudak hareketi üretme eğiliminde olduğunu göstermektedir.

Nihai eğitimde kullanılan veri seti “data/processed/beat_vocarig_1h.npz” dosyasıdır. Eğitim bölmesinde 3.471 sequence ve 129 utterance, doğrulama bölmesinde ise 948 sequence ve 14 utterance bulunmaktadır. Train ve validation utterance'ları arasında çakışma olmaması önemlidir. Bu durum, doğrulama sonucunun aynı konuşma parçalarının ezberlenmesinden değil, ayrılmış konuşmalar üzerindeki performanstan geldiğini gösterir.

Final epochta teacher forcing oranı 0 seviyesine düşmüştür. Bu, modelin son aşamada kendi önceki tahminlerini kullanarak ilerlediği anlamına gelir. Canlı dudak senkronizasyonu için bu davranış önemlidir; çünkü gerçek kullanımda model her karede ideal hedef önceki dudak değerini değil, kendi ürettiği önceki durumu kullanır.

Loss bileşenleri incelendiğinde final validation pose/delta loss 0.00425, velocity loss 0.00169, jerk loss 0.00329, range loss 0.00013 ve silence loss 0.00007 olarak görülmektedir. Pose/delta loss ağız pozunun hedefe yakınlığını, velocity ve jerk loss hareket geçişlerinin yumuşaklığını, range loss blendshape değerlerinin beklenen aralıkta kalmasını, silence loss ise sessiz bölgelerde gereksiz ağız hareketinin azaltılmasını temsil eder. Bu nedenle nihai model yalnızca tek bir toplam loss değeriyle değil, dudak pozisyonu, geçiş sürekliliği, değer aralığı ve sessizlik davranışı birlikte değerlendirilerek ölçülmüştür.

Bu metrikler VocaRig modelinin BEAT tabanlı hazırlanmış veri üzerinde dudak ve çene kanallarını öğrenebildiğini göstermektedir. Ancak dudak senkronizasyonunda sayısal kayıp tek başına yeterli kalite ölçütü değildir. Model düşük validation loss üretse bile avatar üzerinde ağız açıklığı, dudak kapanması, sessizlikte ağız durumu ve konuşma enerjisiyle hareket uyumu ayrıca gözlenmelidir. Bu nedenle nihai değerlendirme, metrik dosyası ve eğitim grafiğinin yanında arayüzdeki 3B avatar önizlemesiyle tamamlanmalıdır.

5.5. Bulguların Değerlendirilmesi

Elde edilen sonuçlar, VocaRig modelinin sentetik ve hazırlanmış veri üzerinde dudak kontrol değerlerini öğrenebildiğini göstermektedir. Modelin 200 epoch boyunca eğitilmiş olması ve CUDA üzerinde çalışması, eğitim sürecinin kısa bir deneme değil, nihai modele yönelik ciddi bir eğitim olduğunu gösterir.

Ancak metrikler tek başına görsel kaliteyi tamamen açıklamaz. Dudak senkronizasyonunda sayısal kayıp kadar görsel algı, konuşma enerjisiyle ağız açıklığı arasındaki uyum, sessizlikte ağız kapanması ve ani geçişlerin yumuşaklığı da önemlidir. Bu nedenle UI içinde 3B viewport, lip barları, waveform ve ARKit tablo çıktısı bulunması doğrulama açısından değerlidir.

Testler sistemin ana sözleşmelerini doğrulasa da, gelecekte kullanıcı tabanlı görsel değerlendirme, gerçek konuşmacı verisiyle çapraz test ve farklı avatar mesh'leriyle uyumluluk testleri eklenmelidir.

5.6. Kabul Senaryoları

Bir projenin doğru çalıştığını göstermek için yalnızca birim testler yeterli değildir. Kullanıcı açısından uçtan uca senaryolar da tanımlanmalıdır. VocaRig için kabul senaryoları, sistemin gerçek kullanım biçimlerini temsil eder. Aşağıdaki senaryolar proje tesliminde kontrol edilmesi gereken davranışları özetler.

Çizelge 5.3. Kabul senaryoları

Senaryo	Adımlar	Beklenen sonuç
Varsayılan sesle çıkartım	UI açılır, voice-sample.wav ile baked çalıştırılır	Lip barları, ARKit tablo ve viewport güncellenir
No-baked WAV akışı	WAV dosyası oynatılırken chunk çıkartımı yapılır	Kareler oynatımla birlikte gelir, UI donmaz
Canlı mikrofon testi	Real test başlatılır, kullanıcı konuşur	Enerji ve ağız değerleri canlı değişir
Sentetik veri üretimi	Pipeline panelinden generate çalıştırılır	Progress güncellenir, .npz ve metadata oluşur
Eğitim başlatma	Dataset seçilir, train start yapılır	Eğitim durumu running olur, metrik eventleri gelir
Model seçimi	Farklı .pt checkpoint seçilir	Aktif model adı değişir ve çıkartım yeni modelle yapılır
ONNX export	Export butonu/API çağırısı kullanılır	.onnx dosyası oluşur ve node bilgisi döner
Benchmark	Benchmark çalıştırılır	Ortalama inference süresi raporlanır

5.7. Değerlendirme Ölçütleri

VocaRig gibi bir projede başarı çok boyutludur. Sayısal kayıp değerleri modelin hedef değerlere yaklaşmasını ölçer; fakat görsel kalite, gecikme ve kullanım kolaylığı da en az kayıp değeri kadar önemlidir. Bu nedenle değerlendirme ölçütleri aşağıdaki gibi gruplanabilir.

Çizelge 5.4. Değerlendirme ölçütleri

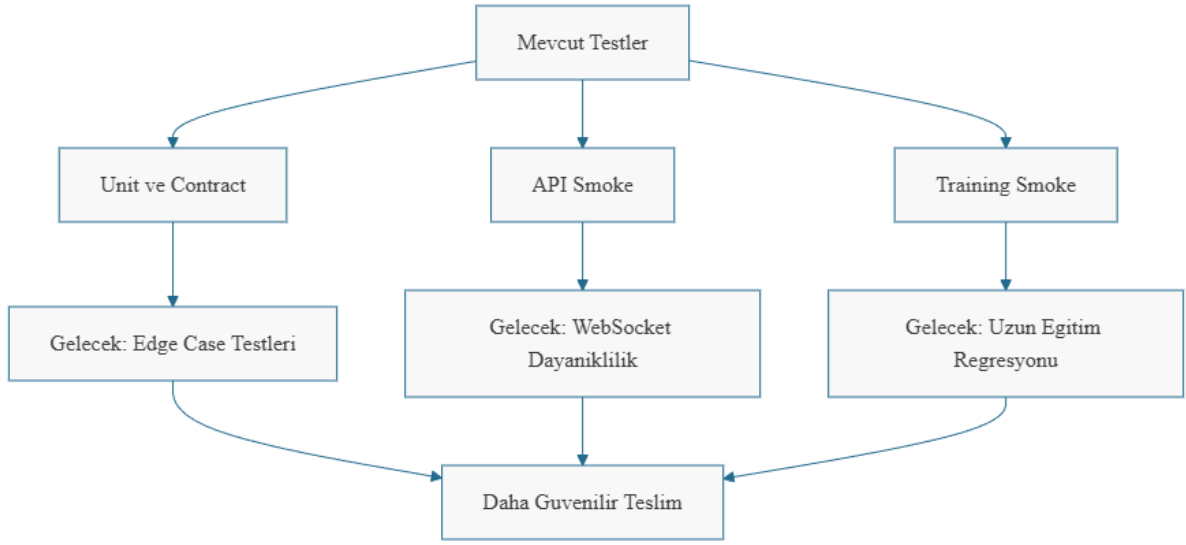
Ölçüt	Açıklama	Ölçüm biçimi
Doğrulama kaybı	Modelin validation setindeki hatası	Metrics JSON
Eğitim kaybı	Modelin train setindeki hatası	Metrics JSON
Gecikme	Bir kare çıkarım süresi	infer_ms, benchmark
Sessizlik davranışı	Sessiz bölgede ağız kapanması	Silence cleanup testleri
Aralık uyumu	Blendshape değerlerinin 0-1 aralığı	Contract ve model testleri
Görsel doğrallık	Avatar üstünde algılanan uyum	UI gözlemi ve kullanıcı değerlendirme
Taşınabilirlik	Modelin farklı runtime'a aktarılabilirliği	ONNX export

Bu ölçütler birlikte kullanıldığında daha doğru bir değerlendirme yapılır. Örneğin yalnızca düşük validation loss elde etmek yeterli değildir; model sessiz bölgelerde ağız hareketi üretiyorsa kullanıcı algısı bozulabilir. Benzer şekilde model doğru çıktı üretse bile inference süresi yüksekse gerçek zamanlı kullanım zayıflar.

5.8. Testlerin Genişletilmesi İçin Öneriler

Mevcut testler projenin temel davranışlarını iyi kapsar. Yine de gelecekte şu testlerin eklenmesi önerilir:

- Farklı örnekleme hızlarındaki WAV dosyalarıyla çıkarım testi.
- Uzun ses dosyalarında bellek ve gecikme testi.
- CUDA yokken CPU fallback testi.
- Geçersiz model yolu, geçersiz dataset yolu ve bozuk JSON hata testleri.
- WebSocket kopma ve yeniden bağlanma testi.
- ONNX modelinin ayrı bir runtime ile yüklenebilirlik testi.
- Gerçek avatar mesh'i üzerinde görsel regresyon testi.
- Farklı volume_threshold değerlerinin sessizlik davranışına etkisini ölçen test.



Şekil 5.4. Test genişletme önerileri

6. SONUÇ

6.1. Genel Sonuç

VocaRig, ses girdisinden ARKit uyumlu dudak ve çene animasyonu üretmeyi hedefleyen bütünlüklü bir yapay sinir ağı projesidir. Proje; veri üretimi, ses özellik çıkarımı, GRU tabanlı model eğitimi, gerçek zamanlı çıkarım, web arayüzü, metrik izleme, ONNX dışı aktarımı ve test altyapısını birlikte sunmaktadır. Bu yönüyle yalnızca teorik bir model denemesi değil, çalıştırılabilir ve gözlemlenebilir bir yazılım sistemi ortaya koymaktadır.

Projenin en güçlü yönlerinden biri, konuşma artikülasyonunu genel yüz ifadesinden ayırmasıdır. VocaRig, ağız ve çene kanallarını sahiplenirken, diğer mimik kanallarının FaceRig benzeri bir sistemden gelebileceği bir birleşim modeli sunar. Runtime mixer katmanı bu ayrımın teknik karşılığını sağlar.

Bir diğer güçlü yön, eğitim ve çıkarım sürecinin aynı arayüzde toplanmasıdır. Kullanıcı veri seti üretebilir, model eğitebilir, model seçebilir, çıkarım yapabilir ve metrikleri izleyebilir. Bu yapı geliştirme döngüsünü kısaltır ve model davranışını daha görünür hale getirir.

6.2. Sınırlılıklar

Projenin bazı sınırlılıkları vardır. İlk sınırlılık veri kalitesidir. Sentetik veri kontrollü ve tekrar üretilebilir olsa da, gerçek konuşma ve yüz hareketi çeşitliliğini tamamen temsil edemez ve gerçek veriler için ayrıca tanımlanmış bağımsız bir kalite metriği bulunmamaktadır. Sentetik veri kontrollü ve tekrar üretilebilir olsa da, gerçek konuşma ve yüz hareketi çeşitliliğini tamamen temsil edemez. Gerçek veriler için ayrıca tanımlanmış bağımsız bir kalite metriği bulunmadığından, bu veriler üzerindeki değerlendirme ağırlıklı olarak görsel gözlem ve arayüz üzerinden yapılan nitel analizlere dayanmaktadır.

İkinci sınırlılık, modelin yalnızca 21 dudak/çene kontrolüne odaklanmasıdır. Bu doğru bir tasarım tercihi olsa da tam yüz animasyonu için ek sistemlerle entegrasyon gerekir.

Üçüncü sınırlılık, gerçek zamanlı kullanımda donanım farklarının sonucu etkilemesidir. CUDA destekli sistemlerde gecikme düşük olabilirken CPU üzerinde çıkarım daha yavaş olabilir.

Dördüncü sınırlılık, görsel kalite değerlendirmesinin yalnızca sayısal kayıpla yapılamamasıdır. Dudak senkronizasyonunda insan algısı önemli olduğundan kullanıcı çalışmaları ve avatar üstü değerlendirme gereklidir.

6.3. Gelecek Çalışmalar

Gelecekte proje şu yönlerde geliştirilebilir:

- Gerçek konuşmacı ve yüz hareketi eşleşmeleriyle daha büyük veri seti oluşturulabilir.
- Türkçe fonem-viseme eşleşmeleri daha ayrıntılı hale getirilebilir.
- Model mimarisi GRU dışında temporal convolution veya transformer tabanlı alternatiflerle karşılaştırılabilir.
- ONNX çıktısı farklı oyun motorlarında test edilebilir.
- Unity veya Unreal Engine entegrasyon katmanı geliştirilebilir.
- UI içinde otomatik görsel kalite raporu ve gecikme analizi eklenebilir.
- Farklı avatar mesh'leriyle ARKit blendshape uyumluluğu test edilebilir.
- Kullanıcı algı çalışmasıyla model çıktılarının doğallığı ölçülebilir.

Sonuç olarak VocaRig, konuşma sesinden gerçek zamanlı dudak animasyonu üretme problemi için uygulanabilir, test edilebilir ve geliştirilebilir bir temel sunmaktadır. Proje, yapay sinir ağı eğitimini pratik bir animasyon üretim hattıyla birleştirdiği için hem akademik hem de uygulamalı yazılım geliştirme açısından değer taşımaktadır.

KAYNAKLAR

- [1] Python Software Foundation, "Python 3.14 Documentation."
Available: <https://docs.python.org/3.14/>. [Erişim tarihi: 31-Mayıs-2026].
- [2] S. van der Walt, S. C. Colbert, and G. Varoquaux, "The NumPy array: a structure for efficient numerical computation," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 22-30, 2011.
Available: <https://numpy.org/doc/>. [Erişim tarihi: 31-Mayıs-2026].
- [3] PyTorch Foundation, "PyTorch Documentation."
Available: <https://docs.pytorch.org/docs/stable/index.html>. [Erişim tarihi: 31-Mayıs-2026].
- [4] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems*, 2019.
Available: <https://arxiv.org/abs/1912.01703>. [Erişim tarihi: 31-Mayıs-2026].
- [5] FastAPI, "FastAPI Documentation." Available: <https://fastapi.tiangolo.com/>. [Erişim tarihi: 31-Mayıs-2026].
- [6] Uvicorn, "Uvicorn Documentation."
Available: <https://www.uvicorn.org/>. [Erişim tarihi: 31-Mayıs-2026].
- [7] ONNX, "Open Neural Network Exchange Documentation."
Available: <https://onnx.ai/onnx/repo-docs/Overview.html>. [Erişim tarihi: 31-Mayıs-2026].
- [8] three.js, "three.js Documentation."
Available: <https://threejs.org/docs/>. [Erişim tarihi: 31-Mayıs-2026].
- [9] Mermaid, "Mermaid Diagramming and Charting Tool."
Available: <https://mermaid.js.org/>. [Erişim tarihi: 31-Mayıs-2026].
- [10] Apple Inc., "ARFaceAnchor.BlendShapes."
Available: <https://developer.apple.com/documentation/arkit/arfaceanchor/blendshapes>. [Erişim tarihi: 31-Mayıs-2026].
- [11] Pydantic, "Pydantic Documentation."
Available: <https://pydantic.dev/docs/>. [Erişim tarihi: 31-Mayıs-2026].
- [12] YAML, "The Official YAML Web Site."
Available: <https://yaml.org/>. [Erişim tarihi: 31-Mayıs-2026].
- [13] Dragonboots Studios, "MetaHuman Head - 52 blendshapes - ARkit ready (Free)".

Available: <https://dragonboots.gumroad.com/l/metahumanhead>. [Erişim tarihi: 31-Mayıs-2026].

[14] Sample-Files, “Sample WAV Audio Files (Free)”.

Available: <https://sample-files.com/audio/wav/>. [Erişim tarihi: 31-Mayıs-2026].

[15] BEAT, “ARkit Data Set (Apache License 2.0)”.

Available: <https://huggingface.co/datasets/H-Liu1997/BEAT>. [Erişim tarihi: 31-Mayıs-2026].